

GIORNO 09

Errori ed eccezioni: il programma che non si ferma alla prima riga sbagliata

Cosa saprete fare stasera

Leggere un traceback dal basso: tipo, messaggio, riga e catena delle chiamate

Scrivere un try/except che cattura solo l'errore previsto, stretto sulla sua riga

Riconoscere le eccezioni già incontrate, e dire quali sono dati e quali difetti

Scegliere fra prevenire con un if e gestire con un try, con un input concreto

Leggere un file senza fermarsi alla riga malformata: scartarla, contarla, registrarla

Far sollevare alle vostre funzioni un errore con un messaggio che dice che cosa non va,
e lasciare a chi chiama la decisione su che cosa fare

Distinguere assert, che controlla il programma, dalla validazione, che controlla i dati

Nei giorni scorsi una frase è tornata più volte in coda a un traceback:

esiste uno strumento per gestire l'errore dopo che è successo. È oggi,
e si usa sui programmi che avete già scritto voi.

1. RIPASSO E DOMANDE

La difesa preventiva, le promesse aperte, il perimetro di oggi

1.1 with open e Path.exists(): la difesa del Giorno 08

```
from pathlib import Path

percorso = Path("ordini_prova.txt")
percorso.write_text("NS-01;Mouse;4\nNS-02;Tastiera;2\nNS-03;Cuffie;5\n",
                    encoding="utf-8")

# 2. La difesa del Giorno 08: prima si guarda se il file c'è, poi si apre.
if not percorso.exists():
    print(f"[ERRORE] Il file {percorso.name} non esiste.")
else:
    totale_pezzi = 0

    with open(percorso, encoding="utf-8") as f:
        for riga in f:
            codice, prodotto, quantita = riga.strip().split(";")
            totale_pezzi += int(quantita)

            print(f"{codice:<7}{prodotto:<10}{quantita:>3}")

    print(f"Pezzi in tutto: {totale_pezzi}")

# TRABOCCHETTO: exists() copre UN guasto solo, il file assente. Una
# quantita' scritta "venti" passa il controllo senza problemi e ferma il
# programma su int(): exists() guarda il disco, non il contenuto.

# Output:
NS-01  Mouse      4
NS-02  Tastiera   2
NS-03  Cuffie     5

Pezzi in tutto: 11
```

1.1 Che cosa exists() non copre: tre guasti

Tre guasti, e nessuno si controlla con un if sul percorso:

- il contenuto sbagliato: una quantità scritta in lettere

- il file scritto in un'altra codifica

- il file che sparisce fra il controllo e l'apertura

È lo schema della validazione preventiva:

- prima si controlla, poi si usa. Applicato al disco invece che alla tastiera.

E ha lo stesso limite: si controlla solo ciò che si è previsto.

1.2 L'anteprima: il ciclo che arriva in fondo

```
# Quattro dati. Nei giorni scorsi tre di questi avrebbero fermato il
# programma al primo giro sbagliato, e il quarto non sarebbe mai stato letto.
DATI = ["72.5", "venti", "12,50", ""]

for testo in DATI:
    try:
        numero = float(testo)
        print(f"[OK] {testo} -> {numero}")
    except ValueError as e:
        # Il messaggio che prima chiudeva il traceback, adesso e' un dato
        # che il programma stampa e poi continua.
        print(f"[ERRORE] {type(e).__name__}: {e}")

# TRABOCCHETTO: l'ultima riga del ciclo si stampa per il dato vuoto, ma il
# messaggio finisce con due apici vuoti. Non e' un refuso: e' il dato.
print("Il programma è arrivato in fondo.")

# Output:
[OK] 72.5 -> 72.5
[ERRORE] ValueError: could not convert string to float: 'venti'
[ERRORE] ValueError: could not convert string to float: '12,50'
[ERRORE] ValueError: could not convert string to float: ''

Il programma è arrivato in fondo.
```

1.3 Prerequisiti operativi: i cinque che tornano oggi

Dal Giorno	Che cosa serve oggi
03	int() e float() sul testo, e il momento in cui esplodono (§6.3)
04	if / elif / else e la validazione con .isdecimal() (§13.4)
06	funzioni che restituiscono con return, main() che regge il programma
07	.get() con il ripiego, la lista di tuple, il conta-occorrenze
08	with open(..., encoding="utf-8"), Path, json.load

Le parole chiave di oggi sono nuove, il resto è roba vostra. Queste cinque tornano in quasi ogni esempio della giornata: se una vi sembra sfocata, rileggete il paragrafo indicato adesso.

1.3 Quattro righe, quattro costrutti: la verifica rapida

```
import json

def dimezza(numero):
    """Restituisce la meta' del numero ricevuto."""
    return numero / 2

# Giorno 04 $13.4: .isdecimal() risponde True solo se sono TUTTE cifre.
print("42".isdecimal(), "4 2".isdecimal())

# Giorno 06 $5.1: il valore restituito si usa, non si stampa dentro.
print(dimezza(9))

# Giorno 07 $11.1: la chiave che manca non ferma niente, esce il ripiego.
listino = {"NORD": 8.50, "SUD": 9.50}
print(listino.get("ESTERO", "n.d.))

# Giorno 08 cap. 12: loads legge JSON da una stringa, load da un file.
print(json.loads('{"porta": 8080}')["porta"] + 1)

# TRABOCCHETTO: "4 2" non e' fatto di sole cifre: lo spazio in mezzo basta
# a dire False. Lo spazio ai BORDI e' un altro discorso: vedi 8.3.
# Output:
True False

4.5

n.d.

8081
```

1.4 Perimetro: che cosa entra

Entra oggi:

try / except, con uno o più tipi, con as e

else e finally, raise, assert

una sola forma della parola class: class NomeErrore(Exception): pass

1.4 Il perimetro in un blocco: la funzione che solleva

```
# cap. 13: una riga che da' un NOME a un errore. Nient'altro.

class QuantitaNonValida(Exception):

    pass


# cap. 11: la funzione che non sa rispondere SOLLEVA, non stampa.

def converti_quantita(testo):

    """Restituisce la quantita' come int, o solleva QuantitaNonValida."""

    if not testo.isdecimal():

        raise QuantitaNonValida(f"quantità non valida: {testo}")

    return int(testo)
```

Ogni riga porta accanto il capitolo che la spiega: rileggetelo stasera e lo capirete per intero. La parola class qui serve solo a dare un nome: non ha metodi, non ha attributi, non costruisce niente.

1.4 Il perimetro in un blocco: try, except, else, finally

```
RIGHE = ["4", "venti", "2"]

buone = 0

scartate = 0

for testo in RIGHE:

    try:                                # cap. 4: provaci

        quantita = converti_quantita(testo)

    except QuantitaNonValida as e:      # cap. 5: solo QUESTO errore

        scartate += 1

        print(f"[!] {e}")

    else:                               # cap. 10: solo se e' andata bene

        buone += 1

        print(f"[OK] quantità {quantita}")

    finally:                            # cap. 10: in ogni caso

        print(f"[fine] riga '{testo}' esaminata")

# cap. 14: il controllo che il programma fa su SE STESSO, non sui dati.
assert buone + scartate == len(RIGHE), "il conto delle righe non torna"

print(f"Buone {buone}, scartate {scartate}")

# Output:

[OK] quantità 4
[fine] riga '4' esaminata
[!] quantità non valida: venti
[fine] riga 'venti' esaminata
[OK] quantità 2
[fine] riga '2' esaminata

Buone 2, scartate 1
```

2. CHE COS'È UN'ECCEZIONE

Un evento che interrompe il flusso, e un oggetto con un tipo e un messaggio

2.1 Il crash come evento: si ferma alla settima riga

```
from pathlib import Path

percorso = Path("ordini_fornitore.txt")
righe = ["12", "8", "15", "20", "5", "9", "venti", "11", "7", "14"]
percorso.write_text("\n".join(righe) + "\n", encoding="utf-8")

totale = 0

with open(percorso, encoding="utf-8") as f:
    for numero_riga, riga in enumerate(f, 1):
        quantita = int(riga.strip())
        totale += quantita
        print(f"riga {numero_riga:>2}: {quantita:>3}    totale {totale:>3}")

print(f"TOTALE ORDINE: {totale}")

# Commenti in coda, così i numeri di riga del traceback restano quelli veri.
# enumerate(f, 1) numera le righe da uno, come le conta un umano (Giorno 07).
# L'accumulatore cresce riga per riga, e alla settima il ciclo si interrompe:
# le sei righe già sommate restano in memoria, ma nessuno le userà più.
# TRABOCCHETTO: l'ultima print non viene eseguita. Il totale parziale, 69,
# esiste per un istante e sparisce con il programma: sullo schermo non
# c'è nessun totale, nemmeno sbagliato.
```

2.1 Sei giri normali, poi il traceback

```
riga 1: 12   totale 12
riga 2:  8   totale 20
riga 3: 15   totale 35
riga 4: 20   totale 55
riga 5:  5   totale 60
riga 6:  9   totale 69
```

```
Traceback (most recent call last):
```

```
  File "prova.py", line 10, in <module>
```

```
    quantita = int(riga.strip())
```

```
                ^^^^^^^^^^^^^^^^^
```

```
ValueError: invalid literal for int() with base 10: 'venti'
```

Il programma ha lavorato normalmente per sei giri. Al settimo int() ha ricevuto un testo che non sa convertire, e invece di restituire un numero ha sollevato un'eccezione. Il totale 69 non compare da nessuna parte.

2.1 Eccezione e sollevare: i due termini

Un'eccezione è un evento che interrompe il flusso normale del programma:

la riga in corso non può essere completata,
e Python smette di eseguire le righe successive.

Sollevare è il verbo tecnico, in inglese raise:

una funzione solleva un'eccezione quando non può fare quello che le è stato chiesto.

Se nessuno la raccoglie, il programma si ferma e stampa il traceback.

Da oggi, qualcuno la può raccogliere.

2.2 Errore di sintassi: il programma non parte nemmeno

```
print("Prima riga: il programma è partito?")

totale = 10 +

print("Ultima riga")

# L'espressione alla riga 2 e' monca: manca il secondo operando.
# Python legge TUTTO il file prima di eseguirne una riga sola, e qui si
# ferma alla lettura. La prima print non viene eseguita.
# TRABOCCHETTO: nessuna riga stampata non significa "si e' fermato alla
#   prima riga". Significa che non e' mai partito.

# Output:

File "prova.py", line 2
    totale = 10 +
                ^
SyntaxError: invalid syntax
```

2.2 Eccezione: parte, e si ferma durante l'esecuzione

```
print("Prima riga: il programma è partito?")

totale = 10 + "5"

print("Ultima riga")

# La sintassi e' giusta: Python legge il file e comincia a eseguirlo.
# La prima print gira, e solo alla riga 2 l'operazione si rivela
# impossibile: un intero e una stringa non si sommano (Giorno 01 $14.5).
# TRABOCCHETTO: qui la prima riga ESCE, sopra no. E' il modo piu' rapido
# per capire, a colpo d'occhio, con quale dei due errori avete a che fare.
```

Output:

```
Prima riga: il programma è partito?
```

```
Traceback (most recent call last):
```

```
File "prova.py", line 2, in <module>
```

```
    totale = 10 + "5"
```

```
~~~~^~~~~
```

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

2.2 Sintassi, esecuzione, logica: quale si cattura

Errore	Quando si vede	Chi lo segnala	Si cattura?
Sintassi (SyntaxError, IndentationError)	prima dell'avvio: non gira nessuna riga	Python, leggendo il file	no: si corregge il codice
Esecuzione (ValueError, TypeError, ...)	durante: gira tutto fino al punto	Python, eseguendo la riga	sì: è la materia di oggi
Logica	mai: il programma finisce e sbaglia	nessuno	no: si trova con i casi limite (Giorno 04 §15.2)

Le eccezioni sono solo la seconda riga. Il try di oggi non ripara un file scritto male e non si accorge di un calcolo sbagliato: raccoglie ciò che succede MENTRE il programma gira.

2.3 L'eccezione è un oggetto: un tipo e un messaggio

```
# 1. Si crea come qualsiasi altro valore: il nome del tipo, e fra parentesi
#     il messaggio. Nessun crash: e' stata creata, non sollevata.
errore = ValueError("importo non valido: '12,50'")

# 2. type() del Giorno 02 $5 funziona anche qui. __name__ da' il nome del
#     tipo come stringa, senza <class ...> attorno.
print(type(errore))
print(type(errore).__name__)

# 3. Stampato, un oggetto eccezione mostra il suo messaggio.
print(errore)

# 4. Tipo, due punti, messaggio: e' ESATTAMENTE l'ultima riga di un
#     traceback. Adesso la sapete costruire da voi.
print(f"{type(errore).__name__}: {errore}")

# TRABOCCHETTO: creare un'eccezione non ferma niente. Serve raise (qui sotto)
#     per sollevarla: finche' non la sollevate, e' un valore come 42.
print("Il programma prosegue: niente è stato sollevato.")

# Output:
<class 'ValueError'>
ValueError
importo non valido: '12,50'
ValueError: importo non valido: '12,50'

Il programma prosegue: niente è stato sollevato.
```

2.3 Lo stesso oggetto, sollevato con raise

```
errore = ValueError("importo non valido: '12,50'")

print("Prima del raise")

raise errore

# raise prende l'oggetto e lo solleva: da qui in poi e' un'eccezione in
# volo, e senza nessuno che la raccolga il programma si ferma.

# TRABOCCHETTO: confrontate l'ultima riga del traceback con la quarta riga
# dell'output del blocco sopra. Sono identiche: tipo, due punti, messaggio.

# Output:

Prima del raise

Traceback (most recent call last):

  File "prova.py", line 3, in <module>

    raise errore

ValueError: importo non valido: '12,50'
```

La chiave di lettura di tutta la giornata: ogni eccezione ha un tipo e un messaggio. Il tipo dice che genere di problema, e decide chi lo cattura; il messaggio dice quale problema, spesso con il dato esatto fra apici.

2.4 Che cosa si perde: il report scritto a metà

```
from pathlib import Path

percorso = Path("report_ordini.txt")
QUANTITA = ["12", "8", "15", "venti", "11"]

with open(percorso, "w", encoding="utf-8") as report:

    print("REPORT ORDINI", file=report)

    for testo in QUANTITA:

        print(f"quantita: {int(testo):>3}", file=report)

        print(f"scritta la riga con {testo}")

print("Report completato.")

# La modalita' "w" svuota il file APPENA lo apre (Giorno 08 §6.5): il report
# della volta prima e' gia' sparito prima che si scriva la prima riga.
# print(..., file=report) scrive sul disco invece che sullo schermo.
# TRABOCCHETTO: "Report completato." non esce, ma il file NON e' rimasto
# vuoto. Contiene l'intestazione e tre righe: vedi il blocco qui sotto.
```

2.4 Tre righe sullo schermo, poi il traceback

```
scritta la riga con 12
scritta la riga con 8
scritta la riga con 15
Traceback (most recent call last):
  File "prova.py", line 8, in <module>
    print(f"quantita: {int(testo):>3}", file=report)
                        ^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'venti'
```

Report completato non esce. Ma che cosa è rimasto sul disco? Per guardarlo serve un blocco che sopravviva al crash: nella slide dopo usiamo in anticipo, senza spiegarlo, lo strumento del capitolo 4.

2.4 Il file troncato che sembra completo

```
from pathlib import Path

percorso = Path("report_ordini.txt")

QUANTITA = ["12", "8", "15", "venti", "11"]

try:

    with open(percorso, "w", encoding="utf-8") as report:

        print("REPORT ORDINI", file=report)

        for testo in QUANTITA:

            print(f"quantita: {int(testo):>3}", file=report)

except ValueError:

    print("[qui il programma di sopra si è fermato]")

# Si rilegge quello che e' rimasto sul disco. Il with, uscendo per l'errore,
# ha chiuso il file e ci ha scritto tutto quello che aveva in sospeso.

contenuto = percorso.read_text(encoding="utf-8")

print(contenuto, end="")

righe_di_dati = len(contenuto.splitlines()) - 1

print(f"Righe di dati sul disco: {righe_di_dati} su {len(QUANTITA)}")

# TRABOCCHETTO: il file ha un aspetto del tutto normale. Chi lo apre domani
# vede un report di tre righe e non ha nessun modo di sapere che ne
# mancano due. Un file troncato e' peggio di nessun file.
```

2.4 Due perdite: la memoria e il disco

[qui il programma di sopra si è fermato]

REPORT ORDINI

quantita: 12

quantita: 8

quantita: 15

Righe di dati sul disco: 3 su 5

In memoria: il totale che l'accumulatore stava costruendo sparisce senza traccia. Sul disco: un report troncato che SEMBRA completo. La seconda è quella che fa danni, perché nessuno se ne accorge. Il capitolo 15 mostra il programma che, quando scarta, lo dice.

3. LEGGERE UN TRACEBACK

Dal basso: tipo, messaggio, punto preciso, riga, catena delle chiamate

```
from pathlib import Path

percorso = Path("dati") / "ordini_ottobre.txt"

print("Apro il file degli ordini...")

with open(percorso, encoding="utf-8") as f:

    righe = f.readlines()

print(f"Righe lette: {len(righe)}")

# Nessun controllo con exists(): e' il programma come lo scrive chi ha
# fretta. La cartella dati/ c'e', il file di ottobre no.

# TRABOCCHETTO: la print alla riga 4 esce, e fa sembrare che il problema
#   sia "dopo". Il traceback dice il contrario: e' la riga 5, l'apertura.

# Output:

Apro il file degli ordini...

Traceback (most recent call last):

File "prova.py", line 5, in <module>

    with open(percorso, encoding="utf-8") as f:

        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

FileNotFoundError: [Errno 2] No such file or directory: 'dati/ordini_ottobre.txt'
```

3.1 Le righe del traceback, dal basso verso l'alto

Riga (dal basso)	Che cosa dice	Qui
ultima, prima dei due punti	il tipo dell'eccezione	FileNotFoundError
ultima, dopo i due punti	il messaggio	il file cercato, fra apici
i ^	il punto preciso della riga	la chiamata a open()
la riga di codice ricopiata	che cosa stava facendo	l'apertura del file
File "...", line 5, in <module>	dove: file, riga, funzione	riga 5, fuori da ogni funzione
Traceback (most recent call last)	l'annuncio: "segue il percorso"	—

Le righe File sono la pila delle chiamate, dalla più esterna in alto alla più interna in basso. <module> è il codice fuori da ogni funzione. Il messaggio ripete il percorso esatto cercato: leggerlo ad alta voce risolve metà dei casi.

3.2 Traceback a più livelli: tre funzioni annidate

```
def converti(testo):  
    return int(testo)  
  
def carica(righe):  
    numeri = []  
    for riga in righe:  
        numeri.append(converti(riga))  
    return numeri  
  
def media(righe):  
    numeri = carica(righe)  
    return sum(numeri) / len(numeri)  
  
print("Media del turno:", media(["72", "85", "sessantaquattro"]))  
  
# Tre livelli di composizione (Giorno 06 §7.1): chi legge il main vede una  
# riga sola, e il lavoro vero sta due funzioni piu' in basso.  
# L'errore nasce in converti(), la funzione piu' interna, e da li' risale:  
# converti -> carica -> media -> modulo, finche' arriva in cima.  
# TRABOCCHETTO: "Media del turno:" non si stampa. Gli argomenti di print  
# si calcolano PRIMA di stampare, e il secondo argomento non arriva mai.
```

Con le funzioni del Giorno 06 la pila si allunga: media() chiama carica(), che chiama converti() per ogni dato.

3.2 Una riga File per ogni chiamata attraversata

```
Traceback (most recent call last):
  File "prova.py", line 17, in <module>
    print("Media del turno:", media(["72", "85", "sessantaquattro"]))
                                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "prova.py", line 13, in media
    numeri = carica(righe)
            ^^^^^^^^^^^^^
  File "prova.py", line 8, in carica
    numeri.append(converti(riga))
                  ^^^^^^^^^^^^^
  File "prova.py", line 2, in converti
    return int(testo)
           ^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'sessantaquattro'
```

Quattro righe File, quattro livelli. È lo stesso traceback a due righe File del Giorno 06 §7.4, con due livelli in più.

3.2 Come si legge una pila di chiamate

1. In fondo, il cosa: ValueError sul testo 'sessantaquattro'.
2. L'ultima riga File è dove l'eccezione è nata:
riga 2, dentro converti. È la funzione più interna, quella che ha fatto il lavoro.
3. Salendo, chi l'aveva chiamata:
carica alla riga 8, media alla riga 13, e in cima il modulo alla riga 17.

L'eccezione è nata in basso ed è risalita attraverso tutte le chiamate.

Questo percorso si chiama propagazione: il capitolo 12 lo usa per decidere a che livello raccoglierla.

3.3 Il sintomo lontano dalla causa: il return mancante

```
def prezzo_scontato(prezzo, sconto):  
    finale = prezzo * (1 - sconto)  
  
    print(f"Prezzo scontato calcolato: {finale:.2f}")  
  
  
def totale_riga(prezzo, sconto, quantita):  
    unitario = prezzo_scontato(prezzo, sconto)  
  
    return unitario * quantita  
  
  
print("Totale:", totale_riga(20.0, 0.10, 3))  
  
# TRABOCCHETTO: il traceback punta alla riga 8 e la riga 8 e' giusta.  
# Il difetto e' la riga 3: una print dove serviva un return.  
  
# Output:  
Prezzo scontato calcolato: 18.00  
Traceback (most recent call last):  
  File "prova.py", line 11, in <module>  
    print("Totale:", totale_riga(20.0, 0.10, 3))  
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
  File "prova.py", line 8, in totale_riga  
    return unitario * quantita  
    ~~~~~^~~~~~  
TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'
```

Quando in un `TypeError` compare `'NoneType'`, la domanda non è che cosa c'è di sbagliato in questa riga, ma da dove arriva quel `None`. Nove volte su dieci, una funzione che non restituisce. Il traceback dice dove guardare per primo, non dove correggere.

3.4 Il valore fra apici: il dato esatto, a capo compreso

```
from pathlib import Path

Path("quantita.txt").write_text("12\nventi\n", encoding="utf-8")

with open("quantita.txt", encoding="utf-8") as f:

    for riga in f:

        print("Letta:", riga.strip())

        numero = int(riga)

# La print mostra la riga ripulita, la conversione riceve quella grezza.
# int() tollera gli spazi e l'a capo ai bordi: sul "12\n" non protesta.
# TRABOCCHETTO: il messaggio mostra 'venti\n', con l'a capo fra gli apici.
#   Non e' lui la causa del crash, ma vi dice che nessuno ha fatto strip()
#   su quel dato: la print vi aveva mostrato un'altra cosa.

# Output:
Letta: 12
Letta: venti
Traceback (most recent call last):

  File "prova.py", line 7, in <module>

    numero = int(riga)
              ^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'venti\n'
```

In fondo ai messaggi di conversione c'è sempre un pezzo fra apici. Non è decorazione: è il dato esatto che la funzione ha ricevuto, carattere per carattere, compresi quelli che a schermo non si vedono.

3.4 Cinque input, cinque messaggi: guardate le ultime parole

```
DIGITATI = ["venti", "", " ", "35 anni", "12,50"]

for testo in DIGITATI:
    try:
        int(testo)
    except ValueError as e:
        print(e)

# TRABOCCHETTO: la seconda e la terza riga sembrano uguali e non lo sono.
#   '' e l'invio a vuoto (Giorno 03 $3.5); ' ' e' uno spazio solo, cioè'
#   uno strip() che mancava. Fra apici la differenza si vede, a occhio no.

# Output:
invalid literal for int() with base 10: 'venti'
invalid literal for int() with base 10: ''
invalid literal for int() with base 10: ' '
invalid literal for int() with base 10: '35 anni'
invalid literal for int() with base 10: '12,50'
```

Gli input che hanno fatto crollare lo screening di 03.7 più due nuovi, messi in fila. Il try serve solo a raccogliere i messaggi uno dopo l'altro: si spiega al capitolo 4.

3.4 Che cosa rivela il dato fra apici

Fra apici	Che cosa rivela	La difesa che mancava
'venti'	una parola al posto delle cifre	nessuna pulizia basta: serve rifiutare
"	invio a vuoto	il controllo di presenza (Giorno 04 §13.3)
' '	spazi rimasti attorno	.strip() (Giorno 03 §5.2)
'35 anni'	l'unità di misura scritta nel campo	il prompt che dice «solo il numero»
'12,50'	la virgola italiana	.replace(",", ".") (Giorno 03 §5.4)
'venti\n'	l'a capo di una riga letta da file	.strip() sulla riga (Giorno 08 cap. 5)

Sei forme dello stesso ValueError, sei cause diverse. Il messaggio vi dice già quale difesa mancava: basta leggere le ultime parole invece di fermarsi al tipo.

4. TRY / EXCEPT: RACCOGLIERE L'ERRORE

Provaci, e se va male con questo tipo di errore, fai quest'altro

4.1 Anatomia di try / except: due clausole, due blocchi

```
testo = "venti"

# 1. try, due punti, blocco rientrato: le righe che POTREBBERO sollevare.
try:
    eta = int(testo)

    print("Età registrata:", eta)

# 2. except, il tipo di eccezione, due punti, blocco rientrato: il GESTORE.
#    Gira solo se nel try e' stata sollevata un'eccezione di QUEL tipo.
except ValueError:

    print(f"[ERRORE] '{testo}' non è un numero intero.")

# 3. Dopo il blocco try/except il programma riprende normalmente, in tutti
#    e due i casi.
print("Il programma prosegue.")

# TRABOCCHETTO: un try senza except (o senza finally, cap. 10) non e'
#    Python valido: il file non parte nemmeno, con un SyntaxError che dice
#    expected 'except' or 'finally' block.

# Output:
[ERRORE] 'venti' non è un numero intero.

Il programma prosegue.
```

4.1 Clausola, gestore, allineamento: i termini da fissare

Clausola: ciascuna parte di un'istruzione composta che comincia con una parola

chiave e finisce con i due punti. try: è una clausola, except ValueError: un'altra.

Lo stesso vale per if, elif ed else del Giorno 04.

Gestore, in inglese handler: il blocco di un except,

il codice che gestisce l'eccezione quando arriva.

Allineamento: except sta sulla stessa colonna del suo try,

come else sta sulla colonna del suo if (Giorno 04 §4.2).

Come un if, gira un ramo solo per volta. Con testo = "35" il gestore è saltato

per intero: Età registrata: 35, poi Il programma prosegue.

4.2 Il flusso con try: dove salta e dove riprende

```
for testo in ["42", "venti"]:
    print(f"--- testo = {testo}")
    print("1. prima del try")
    try:
        print("2. nel try, prima di int()")
        numero = int(testo)
        print(f"3. nel try, dopo int(): numero vale {numero}")
    except ValueError:
        print("4. nell'except")
    print("5. dopo il blocco try")

# Con "42": 1, 2, 3, 5. Il gestore non gira.
# Con "venti": 1, 2, 4, 5. La riga 3 NON gira: appena int() solleva, il
# resto del try viene abbandonato e il programma salta al gestore.
# TRABOCCHETTO: il salto e' SOLO in avanti. Dopo l'except non si torna nel
# try a finire il lavoro: si prosegue con quello che viene dopo.
```

4.2 Con 42: 1, 2, 3, 5. Con venti: 1, 2, 4, 5

```
--- testo = 42
1. prima del try
2. nel try, prima di int()
3. nel try, dopo int(): numero vale 42
5. dopo il blocco try
--- testo = venti
1. prima del try
2. nel try, prima di int()
4. nell'except
5. dopo il blocco try
```

È il diagramma di flusso del Giorno 04 §2.2 con un rombo in più: non la condizione è vera?, ma questa riga ha sollevato?. E il salto ricorda il break del Giorno 05 §10.1: si esce da un blocco a metà e si riprende dalla prima riga dopo.

4.3 Il primo try: int("venti") senza crash

```
testo = input("Eta: ").strip()

try:
    eta = int(testo)
    print(f"Eta registrata: {eta}")
except ValueError:
    print(f"[ERRORE] Eta: serve un numero intero, avete scritto '{testo}'")
    print("Screening interrotto: nessun dato registrato.")

# Una sessione:
Eta: venti

[ERRORE] Eta: serve un numero intero, avete scritto 'venti'

Screening interrotto: nessun dato registrato.
```

Lo screening di 03.7, quello che si era bloccato undici volte, riparato sul campo dell'età. Prima la versione vera, che parla con l'utente.

4.3 La variante collaudabile: i valori scritti nel codice

```
# I tre casi che contano: un dato buono con gli spazi, una parola, un
# invio a vuoto. Lo strip() viene PRIMA del try, come in 03.7.

for digitato in [" 29 ", "venti", ""]:

    testo = digitato.strip()

    try:

        eta = int(testo)

        print(f"Eta registrata: {eta}")

    except ValueError:

        print(f"[ERRORE] Eta: serve un numero intero, avete scritto '{testo}'")


# TRABOCCHETTO: il messaggio ripete il dato fra apici, come fa Python.

# Sull'invio a vuoto esce '' ed e' giusto cosi': e' l'unico modo di far
# vedere a chi usa il programma che non ha scritto niente.


# Output:

Eta registrata: 29

[ERRORE] Eta: serve un numero intero, avete scritto 'venti'

[ERRORE] Eta: serve un numero intero, avete scritto ''
```

4.4 Il file che non c'è, catturato: FileNotFoundError

```
from pathlib import Path

CARTELLA_DATI = Path("dati")

for nome in ["ordini_luglio.txt", "ordini_ottobre.txt"]:
    percorso = CARTELLA_DATI / nome
    try:
        with open(percorso, encoding="utf-8") as f:
            righe = f.readlines()
            print(f"[OK] {percorso.name}: {len(righe)} righe")
    except FileNotFoundError:
        print(f"[ERRORE] {percorso.name}: il file non c'è")

# Luglio c'è e si legge. Ottobre non è ancora arrivato: open() solleva,
# il gestore stampa una riga, e il ciclo passa al mese successivo.
# TRABOCCHETTO: la print con [OK] sta DENTRO il try, dopo il with. Se il
# file manca non si stampa, ed è giusto: è il percorso riuscito. Il
# capitolo 10 le darà un posto più pulito, la clausola else.

# Output:
[OK] ordini_luglio.txt: 3 righe
[ERRORE] ordini_ottobre.txt: il file non c'è
```

4.5 Il try largo: il gestore accusa la riga sbagliata

```
CATALOGO = ["Mouse", "Tastiera", "Cuffie"]

testo_peso = "12.5"

try:
    peso = float(testo_peso)

    posizione = CATALOGO.index("Monitor")

    print(f"Peso {peso} kg, prodotto in posizione {posizione}")
except ValueError:
    print(f"[ERRORE] Peso non valido: '{testo_peso}'")

# float("12.5") riesce. Il ValueError arriva dalla riga dopo: "Monitor"
# non e' nel catalogo, e .index() solleva proprio un ValueError (6.5).
# TRABOCCHETTO: il gestore non sa da quale riga arriva l'eccezione. Accusa
#   il peso, che era giusto, e il vero difetto sparisce senza lasciare
#   traccia. Un messaggio d'errore sbagliato e' peggio di un traceback.

# Output:
[ERRORE] Peso non valido: '12.5'
```

4.5 Il try stretto: una riga dentro, non trenta

```
CATALOGO = ["Mouse", "Tastiera", "Cuffie"]

testo_peso = "12.5"

errore = ""

try:
    peso = float(testo_peso)
except ValueError:
    errore = f"Peso non valido: '{testo_peso}'"

if errore:
    print(f"[ERRORE] {errore}")
else:
    print(f"Peso letto: {peso} kg")
    posizione = CATALOGO.index("Monitor")
    print(f"Prodotto in posizione {posizione}")

# TRABOCCHETTO: adesso il programma si ferma, ed e' il risultato giusto.
# Il difetto sul catalogo esce con il suo nome e la sua riga, invece di
# travestirsi da peso non valido.
```

4.5 Adesso il difetto esce con il suo nome

```
Peso letto: 12.5 kg
Traceback (most recent call last):
  File "prova.py", line 14, in <module>
    posizione = CATALOGO.index("Monitor")
                ^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: list.index(x): x not in list
```

Regola: dentro il try solo le righe che possono sollevare l'eccezione che gestite. Segnale del Giorno 06 §2.3: se un try lo dovete scorrere per vedere dove finisce, è troppo lungo.

4.6 La variabile assegnata nel try: può non esistere

```
ETA_PENSIONE = 67
```

```
def anni_alla_pensione(testo):  
    try:  
        eta = int(testo)  
    except ValueError:  
        print(f"[ERRORE] '{testo}' non è un numero")  
    return ETA_PENSIONE - eta  
  
print(anni_alla_pensione("venti"))  
  
# Stesso difetto, dentro una funzione. eta e' LOCALE (e' assegnata nel  
# corpo), e al momento del return non ha nessun valore.  
# TRABOCCHETTO: il messaggio non dice "not defined" ma "cannot access local  
# variable". Chi cerca NameError nel traceback non lo trova: il tipo e'  
# UnboundLocalError, e il rimedio e' lo stesso.
```

4.6 UnboundLocalError: tre righe dopo il gestore

```
[ERRORE] 'venti' non è un numero
Traceback (most recent call last):
  File "prova.py", line 12, in <module>
    print(anni_alla_pensione("venti"))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "prova.py", line 9, in anni_alla_pensione
    return ETA_PENSIONE - eta
    ^^^
UnboundLocalError: cannot access local variable 'eta' where it is not associated with a value
```

Il gestore ha stampato il suo messaggio e il programma è andato avanti, dritto verso la riga che usa eta. È lo stesso nome mancante del ciclo a zero giri del Giorno 05 §4.2.

4.6 Tre rimedi, dal più debole al più pulito

1. Un valore prima del try: eta = 0

Funziona, ma calcola gli anni alla pensione di un neonato:

è il default comodo e sbagliato del Giorno 06 §8.4.

2. Uscire dal gestore con return

la riga che usa eta non viene raggiunta. È il rimedio della slide dopo.

3. La clausola else del capitolo 10

mette le righe del percorso riuscito in un posto dove eta esiste di sicuro.

4.6 Il rimedio: return nel gestore

```
ETA_PENSIONE = 67

def stampa_anni_alla_pensione(testo):
    try:
        eta = int(testo)

    except ValueError:
        print(f"[ERRORE] '{testo}' non è un numero: calcolo saltato")
        return

    print(f"{eta} anni: ne mancano {ETA_PENSIONE - eta} alla pensione")

stampa_anni_alla_pensione("40")
stampa_anni_alla_pensione("venti")

# Il return nudo esce dalla funzione (Giorno 06 §5.3): la riga del calcolo
# si raggiunge solo se int() e' riuscito, cioe' solo se eta esiste.
# TRABOCCHETTO: senza quel return l'output della seconda chiamata sarebbe
# il messaggio d'errore SEGUITO dal traceback del blocco qui sopra.

# Output:
40 anni: ne mancano 27 alla pensione
[ERRORE] 'venti' non è un numero: calcolo saltato
```

Il return nudo non restituisce niente di utile: serve a uscire. Chi arriva alla riga del calcolo ci arriva solo se int() ce l'ha fatta.

5. CATTURARE L'ECCEZIONE GIUSTA

Il tipo dopo `except` è una dichiarazione: che cosa vi aspettate

5.1 except ValueError: dichiarare il tipo che vi aspettate

```
for valore in ["4", "venti", None]:  
    try:  
        quantita = int(valore)  
        print(f"Quantità: {quantita}")  
    except ValueError:  
        print(f"[!] '{valore}' non è una quantità valida")  
  
# "venti" e' un testo che non si converte: ValueError, catturato.  
  
# None non e' nemmeno un testo: int() non sa che farsene, e solleva un  
# TypeError. Il gestore dice ValueError, quindi il TypeError passa oltre.  
# TRABOCCHETTO: non e' un difetto del try, e' il try che fa il suo lavoro.  
# Un None al posto di una quantita' non e' un dato sbagliato: e' un  
# difetto del programma, e deve fermarlo con il suo vero nome.  
  
# Output:  
Quantità: 4  
[!] 'venti' non è una quantità valida  
Traceback (most recent call last):  
  File "prova.py", line 3, in <module>  
    quantita = int(valore)  
                ^^^^^^^^^^^  
TypeError: int() argument must be a string, a bytes-like object or a real number, not 'NoneType'
```

5.2 Più except in fila: tre gestori, tre guasti

```
from pathlib import Path

Path("turno_mattina.txt").write_text("72\n85\n64\n90\n58\n", encoding="utf-8")
Path("turno_pomeriggio.txt").write_text("58\nsettantasette\n", encoding="utf-8")
Path("turno_sera.txt").write_text("", encoding="utf-8")

def media_da_file(nome):
    """Stampa la media dei punteggi del file, o il motivo per cui manca."""
    try:
        with open(nome, encoding="utf-8") as f:
            punteggi = []
            for riga in f:
                punteggi.append(int(riga))
            media = sum(punteggi) / len(punteggi)
            print(f"{nome:<22}media {media:.1f}")
        # Tre gestori, tre guasti diversi, ognuno con UNA sola origine possibile.
    except FileNotFoundError:
        print(f"{nome:<22}[ERRORE] file assente")
    except ValueError:
        print(f"{nome:<22}[ERRORE] punteggio non numerico")
    except ZeroDivisionError:
        print(f"{nome:<22}[ERRORE] nessun punteggio: media impossibile")
```

5.2 Il primo che combaccia vince: un gestore per file

```
for nome in ["turno_mattina.txt", "turno_pomeriggio.txt", "turno_sera.txt",  
            "turno_notte.txt"]:  
    media_da_file(nome)
```

```
# TRABOCCHETTO: il try qui e' largo, e lo e' apposta: ogni tipo puo'  
# nascere da una riga sola (open, int, la divisione). Se due righe  
# potessero sollevare lo stesso tipo, varrebbe la regola del 4.5.
```

```
# Output:
```

```
turno_mattina.txt      media 73.8  
turno_pomeriggio.txt  [ERRORE] punteggio non numerico  
turno_sera.txt         [ERRORE] nessun punteggio: media impossibile  
turno_notte.txt       [ERRORE] file assente
```

5.3 Una tupla di tipi: except (ValueError, TypeError)

```
import json

ERRORI_DI_CONVERSIONE = (ValueError, TypeError)

testo_json = '{"NS-01": "12.5", "NS-02": "dodici", "NS-03": null, "NS-04": 3}'
pesi = json.loads(testo_json)

for codice, valore in pesi.items():
    try:
        peso = float(valore)
        print(f"[OK] {codice}: {peso} kg")
    except ERRORI_DI_CONVERSIONE:
        print(f"[!] {codice}: valore non convertibile ({valore})")

# null del JSON diventa None in Python (Giorno 08 cap. 12), e float(None)
# solleva TypeError, non ValueError. Il 3 intero invece si converte.
# TRABOCCHETTO: scritta senza parentesi, "except ValueError, TypeError:"
#   fino a Python 3.13 e' un errore di sintassi, e solo dalla 3.14 e'
#   ammessa (e mai insieme ad "as e"). Con le parentesi funziona ovunque:
#   scrivetele sempre, il programma gira anche sulla macchina di un altro.

# Output:
[OK] NS-01: 12.5 kg
[!] NS-02: valore non convertibile (dodici)
[!] NS-03: valore non convertibile (None)
[OK] NS-04: 3.0 kg
```

5.4 except ... as e: il messaggio dentro l'eccezione

```
DATI = ["12", "dodici", "12,5"]

for testo in DATI:
    try:
        quantita = int(testo)
        print(f"[OK] {quantita}")
    except ValueError as e:
        # 1. print(e) stampa il messaggio: quello dopo i due punti.
        print(f"[ERRORE] messaggio: {e}")

        # 2. Tipo e messaggio insieme: la riga per il registro degli errori.
        print(f"[ERRORE] registro: {type(e).__name__}: {e}")

# TRABOCCHETTO: e esiste SOLO dentro il gestore. Python lo cancella
# all'uscita dall'except, e usarlo dopo da' un errore: NameError fuori da
# una funzione, UnboundLocalError dentro main(). Se vi serve dopo, copiate
# il messaggio in una variabile vostra, per esempio motivo = str(e).

# Output:
[OK] 12

[ERRORE] messaggio: invalid literal for int() with base 10: 'dodici'
[ERRORE] registro: ValueError: invalid literal for int() with base 10: 'dodici'
[ERRORE] messaggio: invalid literal for int() with base 10: '12,5'
[ERRORE] registro: ValueError: invalid literal for int() with base 10: '12,5'
```

5.4 CHICCA: repr(e) mostra tipo e messaggio insieme

```
try:
    int("venti")

except ValueError as e:
    # Il messaggio contiene 'venti' fra apici singoli: repr passa alle doppie.
    print(repr(e))

# Qui il messaggio non ha apici: repr torna alle singole.
print(repr(ValueError("peso negativo")))

# Output:
ValueError("invalid literal for int() with base 10: 'venti'")

ValueError('peso negativo')
```

print(e) dà il messaggio nudo; repr(e) dà la forma con cui l'eccezione si scriverebbe nel codice, tipo compreso. Se il messaggio contiene un apice singolo, come quasi tutti quelli di conversione, Python lo racchiude fra virgolette doppie per non confondersi.

5.5 except nudo: il refuso travestito da dato

```
DATI = ["12", "8", "15"]

totale = 0

for testo in DATI:
    try:
        totale = totlae + int(testo)
    except:
        print(f"[!] '{testo}' non è un numero valido")

print(f"Totale: {totale}")

# Il refuso produce un NameError a ogni giro. L'except nudo lo cattura
# insieme a tutto il resto, e stampa il suo messaggio: quello sui DATI.
# TRABOCCHETTO: tre numeri perfettamente validi vengono rifiutati, il totale
# esce zero, e il nome del vero problema non compare da nessuna parte.
# Con except ValueError il programma si sarebbe fermato alla prima riga,
# dicendo NameError: name 'totlae' is not defined.

# Output:
[!] '12' non è un numero valido
[!] '8' non è un numero valido
[!] '15' non è un numero valido

Totale: 0
```

Un except senza tipo cattura tutto. Sembra prudente. È il modo più rapido per nascondere un difetto: qui c'è un refuso nel nome della variabile, totlae invece di totale.

5.5 except Exception: un solo posto legittimo, in cima

```
def main():  
    totale = 0  
  
    for testo in ["12", "8"]:  
        totale = totlae + int(testo)  
  
    print(totale)  
  
try:  
    main()  
except Exception as e:  
    # L'ultima rete: non ripara niente, ma dice che cosa e' successo.  
    print(f"[ERRORE INTERNO] {type(e).__name__}: {e}")  
  
    print("Il programma si ferma: segnalatelo a chi lo mantiene.")  
  
# TRABOCCHETTO: la differenza con il blocco del refuso non e' il tipo  
#   catturato, e' il MESSAGGIO. Qui il nome del difetto arriva a schermo;  
#   la' era travestito da dato non valido.  
  
# Output:  
[ERRORE INTERNO] NameError: name 'totlae' is not defined  
  
Il programma si ferma: segnalatelo a chi lo mantiene.
```

6. IL CATALOGO: LE ECCEZIONI CHE AVETE GIÀ INCONTRATO

Dove l'avete vista, il messaggio esatto, la cattura minima, la prevenzione

6.1 ValueError: il tipo giusto con il valore sbagliato

```
TESTI = ["venti", "12.5", "12,50", " 7 "]
```

```
for testo in TESTI:
```

```
    print(f"[{testo}]")
```

```
    try:
```

```
        print(f"int    -> {int(testo)}")
```

```
    except ValueError as e:
```

```
        print(f"int    -> {e}")
```

```
    try:
```

```
        print(f"float -> {float(testo)}")
```

```
    except ValueError as e:
```

```
        print(f"float -> {e}")
```

```
# Due try separati, uno per conversione: con un try solo, il fallimento di
```

```
# int() salterebbe anche la prova di float() (la regola del 4.2).
```

```
# TRABOCCHETTO: "12.5" e' un float valido e un int NON valido. int() non
```

```
# tronca il testo: il troncamento lo fa solo su un numero, int(12.5) -> 12.
```

6.1 Due messaggi da riconoscere: int() e float()

```
[venti]
int    -> invalid literal for int() with base 10: 'venti'
float  -> could not convert string to float: 'venti'

[12.5]
int    -> invalid literal for int() with base 10: '12.5'
float  -> 12.5

[12,50]
int    -> invalid literal for int() with base 10: '12,50'
float  -> could not convert string to float: '12,50'

[ 7 ]
int    -> 7
float  -> 7.0
```

invalid literal for int() with base 10: literal è il testo da convertire, base 10 la numerazione decimale. could not convert string to float è l'altro. La prevenzione era .isdecimal() con .replace() per i decimali (Giorno 04 §13.4-13.5).

6.1 CHICCA: float() capisce l'inglese, e nan non è uguale a sé

```
for testo in ["1e3", "infinity", "nan", "infinito"]:  
    try:  
        print(f"{testo:<9} -> {float(testo)}")  
    except ValueError as e:  
        print(f"{testo:<9} -> {e}")  
  
non_numero = float("nan")  
  
# Nessun confronto riesce, neanche l'uguaglianza con se stesso.  
  
print("nan == nan ?", non_numero == non_numero)  
print("nan > 0 ?", non_numero > 0)  
print("nan < 0 ?", non_numero < 0)  
  
# Output:  
  
1e3      -> 1000.0  
infinity -> inf  
nan      -> nan  
infinito -> could not convert string to float: 'infinito'  
  
nan == nan ? False  
nan > 0 ? False  
nan < 0 ? False
```

6.2 TypeError: l'operazione che quel tipo non ha

```
# 1. Concatenazione fra testo e numero: Giornata 01 $14.5.
```

```
colli = 3
```

```
try:
```

```
    etichetta = "Colli: " + colli
```

```
except TypeError as e:
```

```
    print("1)", e)
```

```
# 2. Il None della funzione senza return: Giornata 06 $6.2.
```

```
def calcola_sconto(totale):
```

```
    sconto = totale * 0.10
```

```
    print(f"Sconto: {sconto:.2f}")
```

```
try:
```

```
    doppio = calcola_sconto(50.0) * 2
```

```
except TypeError as e:
```

```
    print("2)", e)
```

6.2 Il confronto fra tipi diversi, e i tre messaggi

```
# 3. Il confronto fra tipi diversi: Giorno 04 $7.3 e Giorno 07 $8.3.
```

```
try:
```

```
    if 9 < "10":
```

```
        print("nove viene prima")
```

```
except TypeError as e:
```

```
    print("3)", e)
```

```
# TRABOCCHETTO: nella forma 2 la riga "Sconto: 5.00" esce PRIMA del
```

```
# messaggio d'errore. La funzione ha lavorato; e' il suo None, un rigo
```

```
# dopo, a non poter essere moltiplicato.
```

```
# Output:
```

```
1) can only concatenate str (not "int") to str
```

```
Sconto: 5.00
```

```
2) unsupported operand type(s) for *: 'NoneType' and 'int'
```

```
3) '<' not supported between instances of 'int' and 'str'
```

6.3 ZeroDivisionError: la media calcolata su nessun dato

```
from pathlib import Path

Path("ordini_vuoto.txt").write_text("", encoding="utf-8")

importi = []

with open("ordini_vuoto.txt", encoding="utf-8") as f:
    for riga in f:
        importi.append(float(riga))

print(f"Ordini letti: {len(importi)}")

try:
    media = sum(importi) / len(importi)

    print(f"Media: {media:.2f}")
except ZeroDivisionError as e:
    print(f"[!] {type(e).__name__}: {e}")

# Il file c'è e si apre: nessun FileNotFoundError. Il ciclo fa zero giri,
# e sum() e len() rispondono 0 tutti e due. Solo la divisione ha problemi.
# TRABOCCHETTO: il file vuoto non è la causa, è l'occasione. La causa è
# il denominatore, e un if su len() la previene con una riga (8.4).

# Output:
Ordini letti: 0

[!] ZeroDivisionError: division by zero
```

6.4 IndexError e KeyError: posizione e chiave che non esistono

```
ATTESA = ["Rossi Anna", "Bianchi Marco", "Verdi Luca"]

LISTINO = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}

# 1. Indice oltre la fine: tre elementi, indici da 0 a 2 (Giorno 07 §3.2).
try:
    print(ATTESA[3])
except IndexError as e:
    print("1)", e)

# 2. pop() su una lista vuota: la coda che si e' svuotata.
coda = []
try:
    prossimo = coda.pop(0)
except IndexError as e:
    print("2)", e)

# 3. Chiave assente nell'accesso diretto (Giorno 07 §10.3).
try:
    prezzo = LISTINO["ESTERO"]
except KeyError as e:
    print("3)", e)

# TRABOCCHETTO: ATTESA[3] e' il quarto elemento, non il terzo. Il terzo e'
# ATTESA[2]: "il terzo" per un umano e' l'indice 2 per Python.
```

6.4 I tre messaggi, e la prevenzione del Giorno 07

Caso	Output	Prevenzione del Giorno 07
ATTESA[3] su tre elementi	1) list index out of range	len() prima dell'indice
coda.pop(0) su lista vuota	2) pop from empty list	if coda: prima di pop()
LISTINO["ESTERO"]	3) 'ESTERO'	in o .get() prima della chiave (§11.1)

Guardate il terzo messaggio: è solo la chiave, fra apici. Nessuna frase attorno.

6.4 CHICCA: il messaggio di un KeyError è la chiave stessa

```
LISTINO = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}
```

```
for zona in ["SUD", "ESTERO"]:  
    try:  
        print(f"{zona}: {LISTINO[zona]:.2f} euro")  
    except KeyError as e:  
        print(f"[!] Zona sconosciuta: {e}")  
        print(f"[!] Zona sconosciuta: {zona}")  
        # Gli apici fanno parte del testo: 6 lettere, 8 caratteri.  
        print(f"    caratteri nel messaggio: {len(str(e))}")
```

Output:

```
SUD: 9.50 euro  
[!] Zona sconosciuta: 'ESTERO'  
[!] Zona sconosciuta: ESTERO  
    caratteri nel messaggio: 8
```

6.5 ValueError sulle liste: .index() e .remove()

```
CATALOGO = ["Mouse", "Tastiera", "Cuffie"]
```

```
try:
```

```
    posizione = CATALOGO.index("Webcam HD")
```

```
except ValueError as e:
```

```
    print("index ->", e)
```

```
try:
```

```
    CATALOGO.remove("Webcam HD")
```

```
except ValueError as e:
```

```
    print("remove ->", e)
```

```
# TRABOCCHETTO: il messaggio NON nomina il valore cercato. Dice x, non  
#   'Webcam HD': se nel ciclo cercate dieci prodotti, il messaggio non vi  
#   dice quale mancava. Stampate la variabile vostra accanto al messaggio.
```

```
# Output:
```

```
index -> list.index(x): x not in list
```

```
remove -> list.remove(x): x not in list
```

6.6 NameError e UnboundLocalError: il nome che non esiste

```
ORDINI_DI_OGGI = []

for importo in ORDINI_DI_OGGI:
    ultimo = importo

print(f"Ultimo ordine: {ultimo}")

# La lista e' vuota, il ciclo fa zero giri, e l'assegnazione non avviene.
# TRABOCCHETTO: con una lista piena il programma funziona, e nessun
# collaudo "normale" lo rompe. Si rompe il giorno in cui non ci sono
# ordini, cioe' quando nessuno lo sta guardando.

# Output:
Traceback (most recent call last):
  File "prova.py", line 5, in <module>
    print(f"Ultimo ordine: {ultimo}")
    ^^^^^^
NameError: name 'ultimo' is not defined
```

6.7 AttributeError: il metodo chiesto al tipo sbagliato

```
# 1. Il metodo che restituisce None, assegnato (Giorno 07 $4.3).
carrello = ["Mouse"]
carrello = carrello.append("Tastiera")
try:
    carrello.append("Cuffie")
except AttributeError as e:
    print("1)", e)

# 2. Un metodo di stringa chiesto a una tupla (Giorno 07 $13.1).
ordine = ("NS-01", "mouse", 2)
try:
    print(ordine.upper())
except AttributeError as e:
    print("2)", e)

# TRABOCCHETTO: il primo errore NON e' alla riga del try, e' alla riga 3,
#   dove l'uguale ha messo None al posto della lista. Il try la cattura
#   una riga dopo: e' il sintomo lontano dalla causa del 3.3.

# Output:
1) 'NoneType' object has no attribute 'append'
2) 'tuple' object has no attribute 'upper'
```

6.8 Eccezioni da dati ed eccezioni da difetto

Nasce da un dato: si cattura	Nasce da un difetto: si corregge
ValueError di conversione (int, float)	NameError, UnboundLocalError
KeyError su una chiave arrivata da fuori	AttributeError su None
FileNotFoundError, PermissionError	gran parte dei TypeError
UnicodeDecodeError, JSONDecodeError	IndentationError, SyntaxError
ZeroDivisionError su dati vuoti	IndexError su un indice calcolato male

A sinistra ciò che il programma non controlla: il testo digitato, il file del fornitore, il disco. A destra ciò che il programmatore controlla. Catturare un'eccezione di destra significa nascondere, che è il contrario di correggerla.

6.8 La stessa tabella, in un blocco da tenere accanto

```
NATURA = {  
    "ValueError": "dato: si cattura",  
    "KeyError": "dato, se la chiave arriva da fuori: si cattura",  
    "FileNotFoundError": "dato: si cattura",  
    "ZeroDivisionError": "dato, se i dati sono vuoti: si cattura",  
    "NameError": "difetto: si corregge",  
    "AttributeError": "difetto: si corregge",  
    "TypeError": "difetto, quasi sempre: si corregge",  
}  
  
# Le eccezioni sono oggetti (2.3): si possono creare e mettere in una lista  
# senza sollevarle, e chiedere a ognuna il nome del suo tipo.  
  
TROVATE = [  
    ValueError("invalid literal for int() with base 10: 'venti'"),  
    KeyError("ESTERO"),  
    FileNotFoundError("ordini_ottobre.txt"),  
    ZeroDivisionError("division by zero"),  
    NameError("name 'totlae' is not defined"),  
    AttributeError("'NoneType' object has no attribute 'append'"),  
    TypeError("unsupported operand type(s) for *: 'NoneType' and 'int'"),  
    IndexError("list index out of range"),  
]
```

6.8 La domanda a ogni eccezione: dato o difetto?

```
for errore in TROVATE:

    nome = type(errore).__name__

    print(f"{nome:<18}{NATURA.get(nome, 'da valutare caso per caso')}")


# TRABOCCHETTO: IndexError e' l'unico senza risposta fissa. Un pop(0) su
# una coda svuotata da altri e' un dato; ATTESA[3] su tre elementi e' un
# difetto. Il tipo non basta: conta da dove arriva l'indice.


# Output:

ValueError          dato: si cattura

KeyError            dato, se la chiave arriva da fuori: si cattura

FileNotFoundError  dato: si cattura

ZeroDivisionError  dato, se i dati sono vuoti: si cattura

NameError           difetto: si corregge

AttributeError      difetto: si corregge

TypeError           difetto, quasi sempre: si corregge

IndexError          da valutare caso per caso
```

7. LA GERARCHIA DELLE ECCEZIONI

Famiglie di eccezioni: un except sul genitore cattura anche i figli

7.1 Famiglie di eccezioni: l'albero

BaseException

```
+-- KeyboardInterrupt      il Ctrl+C (7.5)
+-- Exception               tutto quello che si cattura di solito
    +-- ArithmeticError
    |   +-- ZeroDivisionError
+-- LookupError
    |   +-- IndexError
    |   +-- KeyError
+-- OSError                 tutto cio' che riguarda il disco (7.3)
    |   +-- FileNotFoundError
    |   +-- PermissionError
    |   +-- IsADirectoryError
+-- ValueError
    |   +-- UnicodeDecodeError (7.4)
    |   +-- json.JSONDecodeError (7.4)
+-- NameError
    |   +-- UnboundLocalError
+-- TypeError
+-- AttributeError
```

IndexError e KeyError sono due errori dello stesso genere: avete cercato qualcosa in un contenitore, per posizione o per chiave, e non c'era. Ogni tipo ha un genitore più generale, e un except sul genitore cattura anche tutti i figli.

7.1 issubclass: interrogare la gerarchia

```
ATTESA = ["Rossi Anna", "Bianchi Marco", "Verdi Luca"]

LISTINO = {"NORD": 8.50, "SUD": 9.50}

# 1. Le domande alla gerarchia: True significa "e' uno dei figli".
print(issubclass(IndexError, LookupError), issubclass(KeyError, LookupError))
print(issubclass(ZeroDivisionError, ArithmeticError))
print(issubclass(UnboundLocalError, NameError))
print(issubclass(ValueError, LookupError))

# 2. Un gestore sul genitore raccoglie tutti e due i figli.
for prova in ["posizione", "chiave"]:
    try:
        if prova == "posizione":
            valore = ATTESA[5]
        else:
            valore = LISTINO["ESTERO"]
    except LookupError as e:
        # type(e) dice il tipo VERO, anche se il gestore e' sul genitore.
        print(f"{prova}: {type(e).__name__}, catturato come LookupError")

# TRABOCCHETTO: ValueError NON e' un LookupError, anche se .index() su una
# lista (6.5) e' a sua volta una ricerca fallita. La famiglia la decide
# chi ha scritto Python, non il buon senso: in caso di dubbio, chiedete a
# issubclass.
```

7.1 Il tipo vero, catturato come genitore

```
True True
True
True
False
posizione: IndexError, catturato come LookupError
chiave: KeyError, catturato come LookupError
```

Le prime quattro righe sono le risposte della gerarchia; le ultime due mostrano un gestore solo, scritto sul genitore, che raccoglie due figli diversi. `type(e)` continua a dire il tipo vero.

7.2 Ordine degli except: il generale prima zittisce gli altri

```
for testo in ["12", "dodici"]:
    try:
        quantita = int(testo)
        print(f"[OK] {quantita}")
    except Exception:
        print(f"[ERRORE GENERICO] qualcosa non va con '{testo}'")
    except ValueError:
        print(f"[ERRORE] '{testo}' non è un numero intero")

# ValueError e' un figlio di Exception: il primo gestore combaccia, e i
# gestori si provano dall'alto (5.2). Il secondo non e' raggiungibile.
# TRABOCCHETTO: Python non dice niente. Nessun errore, nessun avviso: il
# messaggio preciso che avete scritto e' codice morto, e il programma
# stampa per sempre quello generico.

# Output:
[OK] 12
[ERRORE GENERICO] qualcosa non va con 'dodici'
```

7.3 OSError e le sue figlie: il disco

```
from pathlib import Path

# Una cartella vera, creata come nel Giorno 08 $8.3.
Path("archivio").mkdir(exist_ok=True)

for nome in ["archivio", "manca.txt"]:
    try:
        with open(nome, encoding="utf-8") as f:
            print(f.read())
    except FileNotFoundError:
        print(f"{nome}: il file non esiste")
    except OSError:
        # Il messaggio non nomina il tipo: su sistemi diversi e' diverso.
        print(f"{nome}: esiste, ma non si può aprire come file")

print(issubclass(IsADirectoryError, OSError), issubclass(PermissionError, OSError))

# TRABOCCHETTO: aprire una cartella come file solleva IsADirectoryError su
# macOS e Linux, e PermissionError su Windows. Un except IsADirectoryError
# funzionerebbe sul vostro portatile e fallirebbe su quello del collega.

# Output:
archivio: esiste, ma non si può aprire come file
manca.txt: il file non esiste

True True
```

7.4 UnicodeDecodeError e JSONDecodeError sono ValueError

```
import json

from pathlib import Path

NOMI = ["config_sportello_rotto.json", "ordini_giugno_latin1.txt"]

for nome in NOMI:
    percorso = Path("dati") / nome

    try:
        with open(percorso, encoding="utf-8") as f:
            testo = f.read()

            if percorso.suffix == ".json":
                dati = json.loads(testo)

                print(f"[OK] {nome}")
    except ValueError as e:
        print(f"[ERRORE] {nome}: {type(e).__name__}")
        print(f"    {e}")

# Il JSON si legge senza problemi come testo, e si rompe in json.loads.
# Il file latin-1 si rompe gia' in read(): la e accentata, scritta in
# latin-1, e' un byte che in utf-8 non puo' stare dove sta.
# TRABOCCHETTO: un solo except ValueError, e due guasti di natura diversa.
# Se nel try ci fosse anche un int() sui campi, un numero scritto male
# finirebbe nello stesso gestore: il try stretto del 4.5 vale doppio qui.
```

7.4 Riga e colonna, byte e posizione

```
[ERRORE] config_sportello_rotto.json: JSONDecodeError
  Expecting ',' delimiter: line 3 column 3 (char 40)
[ERRORE] ordini_giugno_latin1.txt: UnicodeDecodeError
  'utf-8' codec can't decode byte 0xe8 in position 44: invalid continuation byte
```

Il messaggio del JSON dice riga e colonna del difetto; quello della codifica dice il byte e la sua posizione. Il capitolo 9 li usa tutti e due per un messaggio che chi usa il programma possa capire, e per il file latin-1 prova una seconda codifica.

7.4 CHICCA: due errori di file che sono errori di valore

```
import json

# Due figli di ValueError, e un OSError per controprova.
print(issubclass(json.JSONDecodeError, ValueError))

print(issubclass(UnicodeDecodeError, ValueError))

print(issubclass(FileNotFoundError, ValueError))

# Output:

True

True

False
```

È il motivo per cui l'except ValueError di prima li ha presi tutti e due, e il motivo per cui un except ValueError scritto per un int() può raccogliere, senza che ve ne accorgiate, un file illeggibile.

7.5 Exception e BaseException: perché il Ctrl+C sta fuori

```
print(issubclass(KeyboardInterrupt, Exception))

print(issubclass(KeyboardInterrupt, BaseException))

try:

    print("Elaborazione in corso...")

    raise KeyboardInterrupt

except Exception:

    print("[!] errore catturato, riprovo")

print("Questa riga non si stampa.")

# Stesso blocco del 5.5, con except Exception al posto dell'except nudo.

# KeyboardInterrupt non e' un Exception: il gestore non combacia, e

# l'interruzione arriva in cima come deve.

# TRABOCCHETTO: e' l'unica differenza pratica fra "except:" ed "except

#   Exception:", ed e' quella che decide se il vostro programma si ferma

#   quando premete Ctrl+C.

# Output:

False

True

Elaborazione in corso...

Traceback (most recent call last):

  File "prova.py", line 5, in <module>

    raise KeyboardInterrupt

KeyboardInterrupt
```

8. PREVENIRE O GESTIRE: DUE STRADE SUGLI STESSI PROGRAMMI

Lo stesso programma con l'if e con il try: si sceglie sui casi limite

8.1 Prevenire: prima si guarda, poi si salta

```
testo = "venti"

# PREVENIRE: il controllo viene prima, e int() gira solo sul dato sicuro.
if testo.isdecimal():
    print("prevenire:", int(testo))
else:
    print("prevenire: rifiutato prima di convertire")

# Output:
prevenire: rifiutato prima di convertire
```

Si controlla il dato con un if, e lo si usa solo se il controllo passa: è quello che fate dal Giorno 04 §13.1. In inglese LBYL, Look Before You Leap. Questo è il capitolo per cui la giornata esiste: ogni programma, in due versioni affiancate.

8.1 Gestire: si prova, e se va male si rimedia

```
testo = "venti"

# GESTIRE: int() gira subito, e il rifiuto arriva dopo il tentativo.
try:
    print("gestire:", int(testo))
except ValueError:
    print("gestire: la conversione è fallita, rifiutato dopo")

# TRABOCCHETTO: le due strade non controllano la stessa cosa. isdecimal()
# e' una regola scritta da voi; int() e' la conversione vera. Su "venti"
# concordano, sugli input del paragrafo 8.3 no.

# Output:
gestire: la conversione è fallita, rifiutato dopo
```

8.1 Le due strategie a confronto

	Prevenire (if)	Gestire (try)
Che cosa costa	un controllo per ogni guasto previsto	un gestore per ogni tipo di eccezione
Che cosa copre	solo ciò che avete immaginato	tutto ciò che l'operazione rifiuta davvero
Dove si legge la regola	nell'if, esplicita	nella funzione che converte, implicita
Punto debole	il caso non previsto passa, o viene rifiutato a torto	un try largo cattura l'errore sbagliato (§4.5)
Quando il mondo cambia fra controllo e uso	il controllo è già vecchio (§9.2)	non c'è un «fra»: si prova e basta

Sugli input ordinari le due strade danno lo stesso risultato. La differenza sta nei casi limite, ed è lì che si sceglie.

8.2 L'età del Giorno 04, versione if: .isdecimal()

```
MAGGIORE_ETA = 18
```

```
def verifica_con_if(testo):  
    """Prevenire: si controlla la forma, e solo dopo si converte."""  
    testo = testo.strip()  
    if not testo.isdecimal():  
        return "[ERRORE] serve un numero intero"  
    if int(testo) >= MAGGIORE_ETA:  
        return "firma ammessa"  
    return "firma NON ammessa"
```

Lo sportello di 04.1: la firma è ammessa solo dai diciotto anni compiuti. Prima la versione che conoscete, con la regola sulla forma scritta a mano.

8.2 L'età del Giorno 04, versione try: int()

```
def verifica_con_try(testo):  
    """Gestire: si converte, e se la conversione fallisce si rimedia."""  
  
    try:  
        eta = int(testo)  
  
    except ValueError:  
        return "[ERRORE] serve un numero intero"  
  
    if eta >= MAGGIORE_ETA:  
        return "firma ammessa"  
  
    return "firma NON ammessa"  
  
  
print(f"{'input':<8}{ 'con if':<34}con try")  
  
for digitato in ["30", "17", "venti", ""]:  
    etichetta = f"{'digitato}'"  
  
    print(f"{'etichetta':<8}{verifica_con_if(digitato):<34}"  
          f"{'verifica_con_try(digitato)}")  
  
  
# Le due funzioni restituiscono il testo dell'esito invece di stamparlo  
# (Giorno 06 $5.2): così si possono mettere affiancate nella stessa riga.  
# TRABOCCHETTO: nella versione con try non c'è lo strip(), e non serve:  
# int() tollera gli spazi ai bordi da solo. E' la prima delle differenze  
# che il paragrafo 8.3 mette in fila.
```

8.2 Sugli input ordinari, indistinguibili

input	con if	con try
'30'	firma ammessa	firma ammessa
'17'	firma NON ammessa	firma NON ammessa
'venti'	[ERRORE] serve un numero intero	[ERRORE] serve un numero intero
' '	[ERRORE] serve un numero intero	[ERRORE] serve un numero intero

Quattro input che conoscete, quattro risposte identiche nelle due colonne. Il paragrafo dopo cerca gli input su cui le due strade si separano.

8.3 Otto input, due regole: prevenire e gestire

```
CASI = [" 42 ", "-5", "+7", "1_000", "42", "4.2", "quattro", ""]
```

```
print(f"{'input':<10}{'isdecimal()':<14}{'int()':<9}stesso esito?")
```

```
divergenze = 0
```

```
for testo in CASI:
```

```
    # 1. Prevenire: la regola del Giorno 04 $13.4.
```

```
    if testo.isdecimal():
```

```
        esito_if = "accetta"
```

```
    else:
```

```
        esito_if = "rifiuta"
```

```
    # 2. Gestire: la conversione stessa fa da controllo.
```

```
    try:
```

```
        esito_try = str(int(testo))
```

```
    except ValueError:
```

```
        esito_try = "errore"
```

8.3 Otto input, due regole: dove si separano

```
# 3. Concordano se accettano insieme o rifiutano insieme.

accetta_if = esito_if == "accetta"
accetta_try = esito_try != "errore"

if accetta_if == accetta_try:
    accordo = "sì"
else:
    accordo = "NO"
    divergenze += 1

etichetta = f'''{testo}'''

print(f"{etichetta:<10}{esito_if:<14}{esito_try:<9}{accordo}")

print(f"Divergenze: {divergenze} su {len(CASI)}")


# TRABOCCHETTO: "-5" passa int() e diventa -5. Gestire la conversione NON
# esonera dal controllo d'intervallo: un'eta' di meno cinque anni e' un
# intero perfetto (Giorno 04 §13.6).
```

8.3 Quattro divergenze, in direzioni diverse

```
input      .isdecimal()  int()      stesso esito?
' 42 '     rifiuta      42         NO
'-5'       rifiuta      -5         NO
'+7'       rifiuta      7         NO
'1_000'    rifiuta      1000        NO
'42'       accetta      42         sì
'4.2'      rifiuta      errore        sì
'quattro'  rifiuta      errore        sì
''         rifiuta      errore        sì
Divergenze: 4 su 8
```

Due a favore del try: rifiutare " 42 " e "+7" è pignoleria. Una contro: "-5" è un intero, ma un'età negativa no. Nel 06.3 il -3 riceveva «servono cifre», che è falso; con int() arriva al controllo d'intervallo. Ma solo se il controllo c'è.

8.3 CHICCA: int() tollera spazi, segno e trattini bassi

```
print(int(" 1_000 "))
print(int("+7") + int("-5"))
# Anche il tab e l'a capo ai bordi spariscono, come gli spazi.
print(int("\t42\n"))
for testo in ["1__000", "1.000"]:
    try:
        print(int(testo))
    except ValueError as e:
        print(e)

# Output:
1000
2
42
invalid literal for int() with base 10: '1__000'
invalid literal for int() with base 10: '1.000'
```

8.4 La media, versione if: if len(...) == 0

```
def media_con_if(punteggi):  
    """Prevenire: il caso vuoto e' previsto e ha un ramo suo."""  
    if len(punteggi) == 0:  
        return "nessun punteggio: media non calcolabile"  
    return f"{sum(punteggi) / len(punteggi):.1f}"
```

La media dei punteggi di un turno, dal Giorno 05 §6.4 al 07.3. Qui vince l'if, per tre ragioni: il caso è prevedibile, una lista vuota è un evento normale; il controllo costa una riga; e dichiara il caso a chi legge. Forma più corta: if not punteggi:.

8.4 La media, versione try: except ZeroDivisionError

```
def media_con_try(punteggi):  
    """Gestire: si divide, e la divisione per zero si raccoglie dopo."""  
  
    try:  
        return f"{sum(punteggi) / len(punteggi):.1f}"  
    except ZeroDivisionError:  
        return "nessun punteggio: media non calcolabile"  
  
for turno in [[72, 85, 64, 90, 58], []]:  
    print("if :", media_con_if(turno))  
    print("try:", media_con_try(turno))  
  
# Stesso output, riga per riga. La differenza e' in chi legge il codice:  
# l'if DICE che la lista puo' essere vuota, il try lo lascia indovinare.  
# TRABOCCHETTO: se un giorno la riga della media diventa una formula con un  
# secondo denominatore (per esempio i turni lavorati), il try raccoglie  
# anche quella divisione e risponde "nessun punteggio" sul dato sbagliato.  
  
# Output:  
if : 73.8  
try: 73.8  
if : nessun punteggio: media non calcolabile  
try: nessun punteggio: media non calcolabile
```

8.5 Il listino: in, .get() ed except KeyError

```
LISTINO = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}
RIPIEGO = "prezzo da concordare"

for zona in ["SUD", "ESTERO"]:

    # 1. Prevenire con in (Giorno 07 $5.2): si guarda, poi si accede.

    if zona in LISTINO:

        forma_in = f"{LISTINO[zona]:.2f}"

    else:

        forma_in = RIPIEGO

    # 2. .get() con il ripiego (Giorno 07 $11.1): una riga, nessun ramo.

    forma_get = LISTINO.get(zona, RIPIEGO)

    # 3. Gestire: si accede, e la chiave assente si raccoglie dopo.

    try:

        forma_try = f"{LISTINO[zona]:.2f}"

    except KeyError:

        forma_try = RIPIEGO

    print(f"zona:<7)in -> {forma_in}")

    print(f"zona:<7)get -> {forma_get}")

    print(f"zona:<7)try -> {forma_try}")

# TRABOCCHETTO: .get() restituisce il float quando la chiave c'e' e la
# stringa quando manca: la variabile cambia tipo. Per questo esce 9.5 e
# non 9.50, e un :.2f applicato al ripiego si fermerebbe con ValueError:
# Unknown format code 'f' for object of type 'str'.
```

8.5 Ripiego legittimo o anomalia da segnalare

```
SUD    in    -> 9.50
SUD    get   -> 9.5
SUD    try   -> 9.50
ESTERO in    -> prezzo da concordare
ESTERO get   -> prezzo da concordare
ESTERO try   -> prezzo da concordare
```

Il criterio è il significato della chiave assente. Se c'è un ripiego legittimo, una zona non a listino ha davvero un prezzo da concordare: vince .get(). Se è un'anomalia, una spedizione su una zona inventata: vince il try, dove si scrive il messaggio e si conta lo scarto.

8.6 La coda, versione if: if coda:

```
def chiama_con_if(coda):  
    """Prevenire: si guarda se la coda ha qualcuno, poi lo si chiama."""  
    if coda:  
        return f"Avanti {coda.pop(0)}"  
    return "Nessuno in attesa"
```

La coda del Poliambulatorio Aurora di 07.2: si chiama il primo in attesa, finché ce n'è uno. Qui prevenire è più leggibile: if coda: è la domanda che fa un operatore vero, c'è qualcuno?

8.6 La coda, versione try: except IndexError su pop(0)

```
def chiama_con_try(coda):  
    """Gestire: si chiama, e la coda vuota si raccoglie dopo."""  
  
    try:  
        return f"Avanti {coda.pop(0)}"  
  
    except IndexError:  
        return "Nessuno in attesa"  
  
  
attesa_a = ["Rossi Anna", "Bianchi Marco"]  
attesa_b = ["Rossi Anna", "Bianchi Marco"]  
  
for numero in range(1, 4):  
  
    print(f"{numero}) if : {chiama_con_if(attesa_a):<22}"  
        f"try: {chiama_con_try(attesa_b)}")  
  
  
# Due code identiche e separate: ogni funzione svuota la sua.  
# TRABOCCHETTO: attesa_b = attesa_a darebbe DUE nomi per la stessa lista  
# (Giorno 07 $7.3). La seconda funzione troverebbe la coda gia' svuotata  
# dalla prima, e la seconda colonna sarebbe sbagliata senza nessun errore.  
  
# Output:  
  
1) if : Avanti Rossi Anna      try: Avanti Rossi Anna  
2) if : Avanti Bianchi Marco  try: Avanti Bianchi Marco  
3) if : Nessuno in attesa     try: Nessuno in attesa
```

8.7 Il ciclo di validazione riscritto con try

```
MIN_COMPONENTI = 1
MAX_COMPONENTI = 12
TENTATIVI_MASSIMI = 4
NESSUN_VALORE = -1

def chiedi_intero(risposte, minimo, massimo):
    """Restituisce la prima risposta intera nell'intervallo, o NESSUN_VALORE."""
    for testo in risposte[:TENTATIVI_MASSIMI]:
        print(f"Componenti del nucleo: {testo}")

        try:
            valore = int(testo)

        except ValueError:
            print("[ERRORE] Serve un numero intero.")
            continue

        # Da qui in poi valore e' un int sicuro: la forma e' gia' passata.
        if valore < minimo:
            print(f"[ERRORE] Il valore minimo ammesso è {minimo}.")

        elif valore > massimo:
            print(f"[ERRORE] Il valore massimo ammesso è {massimo}.")

        else:
            return valore

    print("[!] Troppi tentativi: la pratica passa all'operatore.")
    return NESSUN_VALORE
```

8.7 Le risposte simulate: il blocco si collauda

```
for risposte in [ ["tre", "-3", "13", " 4"], ["zero", "0", "99", "sei", "5"] ]:  
    valore = chiedi_intero(risposte, MIN_COMPONENTI, MAX_COMPONENTI)  
    if valore == NESSUN_VALORE:  
        print("Nessun valore registrato.")  
    else:  
        print(f"Valore registrato: {valore}")  
  
# TRABOCCHETTO: senza il continue nel gestore, il programma andrebbe avanti  
#   fino al confronto con minimo usando un valore mai assegnato in questo  
#   giro: al primo "tre" si fermerebbe con UnboundLocalError (4.6).
```

Al posto della tastiera c'è una lista di risposte simulate, così il blocco si collauda. Il continue nel gestore è il pezzo che tiene insieme tutto: salta al giro dopo prima che valore venga usato.

8.7 Due controlli in meno, due messaggi più onesti

```
Componenti del nucleo: tre
[ERRORE] Serve un numero intero.
Componenti del nucleo: -3
[ERRORE] Il valore minimo ammesso è 1.
Componenti del nucleo: 13
[ERRORE] Il valore massimo ammesso è 12.
Componenti del nucleo: 4
Valore registrato: 4
Componenti del nucleo: zero
[ERRORE] Serve un numero intero.
Componenti del nucleo: 0
[ERRORE] Il valore minimo ammesso è 1.
Componenti del nucleo: 99
[ERRORE] Il valore massimo ammesso è 12.
Componenti del nucleo: sei
[ERRORE] Serve un numero intero.
[!] Troppi tentativi: la pratica passa all'operatore.
Nessun valore registrato.
```

Il -3 nel 06.3 riceveva «servono solo cifre», che è falso; qui riceve il minimo ammesso. " 4" passa senza .strip(). La quinta risposta del secondo giro, "5", era buona: ma arriva dopo il tetto, e non viene letta.

8.8 La tabella di decisione: if, try, o tutti e due

Il guasto è...	Esempio di oggi	Strada
prevedibile, economico da controllare	lista vuota, coda vuota	if (§8.4, §8.6)
una chiave assente con un ripiego legittimo	zona non a listino	.get() (§8.5)
una chiave assente che è un'anomalia	zona inventata in una spedizione	try (§8.5)
deciso da un parser (int, float, json)	"+7", "1e3", un JSON rotto	try: la regola è quella del parser
deciso dal mondo esterno (disco, file di altri)	il file che sparisce, i permessi	try: un controllo è già vecchio quando lo usate (§9.2)
un dato scritto da una persona	l'età allo sportello	tutti e due: try per la forma, if per l'intervallo

Nella forma della tabella di decisione del Giorno 07 §15.1: dal tipo di guasto alla strada. L'ultima riga è quella che userete di più.

8.8 Tutti e due: try per la forma, if per l'intervallo

```
ETA_MINIMA = 0
ETA_MASSIMA = 120
```

```
def valida_eta(testo):
    """Restituisce (eta, "") se il dato e' buono, (0, motivo) se non lo e'."""
    # 1. La FORMA la decide int(): e' la regola vera, non una imitazione.
    try:
        eta = int(testo)
    except ValueError:
        return 0, "forma: serve un numero intero"
    # 2. L'INTERVALLO lo decidete voi: nessun parser sa che 150 anni sono
    # troppi. Confronto concatenato del Giorno 04 §10.3.
    if not ETA_MINIMA <= eta <= ETA_MASSIMA:
        return 0, f"intervallo: fra {ETA_MINIMA} e {ETA_MASSIMA}"
    return eta, ""
```

8.8 Forma e intervallo, su quattro input

```
for testo in ["42", "venti", "-5", "1_000"]:
    eta, motivo = valida_eta(testo)

    if motivo:
        print(f"'{testo}': rifiutato ({motivo})")
    else:
        print(f"'{testo}': accettato, {eta} anni")

# TRABOCCHETTO: "1_000" supera la forma (8.3) e cade sull'intervallo. Senza
#   il punto 2, uno sportello accetterebbe un cittadino di mille anni.

# Output:
'42': accettato, 42 anni
'venti': rifiutato (forma: serve un numero intero)
'-5': rifiutato (intervallo: fra 0 e 120)
'1_000': rifiutato (intervallo: fra 0 e 120)
```

9. LE ECCEZIONI DEI FILE: I CASI CHE IL GIORNO 08 HA LASCIATO APERTI

Ogni esempio crea da sé il file che legge: si incolla e gira così com'è

9. I casi lasciati aperti, e dove si chiudono

Caso	Dove l'avete incontrato	Che cosa faceva il Giorno 08	Qui
file assente	Giorno 08 §3.4 e §8.1	is_file() prima di aprire	9.1
file che sparisce	Giorno 08 §8.5	solo prosa: nessun if lo vede	9.2
codifica diversa	Giorno 08 §2.4	encoding="utf-8" sempre	9.3
JSON malformato	Giorno 08 §12.6	traceback letto, nessuna difesa	9.4
JSON valido ma sbagliato	Giorno 08 §12.2	fiducia nel file	9.5
riga che non si converte	Giorno 08 §9.2-9.4	len(campi) e .isdecimal()	9.6-9.7

Il Giorno 08 si è chiuso con un elenco: i punti in cui il programma delle spedizioni si sarebbe fermato comunque, per quanti if gli metteste davanti. Questo capitolo prende quei casi e li chiude.

9.1 FileNotFoundError: il messaggio per chi usa il programma

```
from pathlib import Path

Path("colli_nord.txt").write_text("12.5\n3.2\n", encoding="utf-8")
Path("colli_sud.txt").write_text("6.7\n4.8\n22.0\n", encoding="utf-8")

MAGAZZINI = ["colli_nord.txt", "colli_sud.txt", "colli_isole.txt"]

colli_totali = 0

for nome in MAGAZZINI:
    percorso = Path(nome)

    try:
        with open(percorso, encoding="utf-8") as f:
            righe = f.readlines()

    except FileNotFoundError:
        print(f"{percorso.name:<18}{'MANCANTE':>10}  -> richiedetelo al magazzino")
        continue

    colli_totali += len(righe)

print(f"{percorso.name:<18}{len(righe):>10}")

# TRABOCCHETTO: togliete il continue. Il file mancante stampa MANCANTE e
# subito dopo una seconda riga con 3 colli: sono quelli di colli_sud.txt,
# rimasti nella variabile righe dal giro prima. Totale 8 invece di 5, e
# nessun errore da nessuna parte.

print(f"{'Totale colli':<18}{colli_totali:>10}")
```

9.1 Il buco dichiarato nella colonna

```
colli_nord.txt          2
colli_sud.txt           3
colli_isole.txt         MANCANTE  -> richiedetelo al magazzino
Totale colli            5
```

Il traceback del Giorno 08 parla a chi ha scritto il programma; al magazziniere serve sapere quale file manca e che cosa fare. Il report esce, con il buco nella colonna. E non stampa l'eccezione: il suo testo contiene il percorso, che cambia da una macchina all'altra (E9).

9.2 Path.exists() contro try: il file che sparisce

```
from pathlib import Path

percorso = Path("listino_zone.txt")

percorso.write_text("NORD;8.50\nCENTRO;7.00\n", encoding="utf-8")

# 1. La difesa del Giorno 08 §8.1: si controlla prima di aprire.

if percorso.is_file():

    print("Controllo preventivo: il file c'è")

    # 2. Fra controllo e apertura il file sparisce. Nella realtà e' un altro
    # programma a cancellarlo; qui lo facciamo noi, per vederlo succedere.
    percorso.unlink()

    # 3. Il controllo ha detto sì, eppure l'apertura fallisce: questo caso
    # lo raccoglie solo il try.

    try:

        with open(percorso, encoding="utf-8") as f:

            righe = f.readlines()

            print("Righe lette:", len(righe))

    except FileNotFoundError:

        print("Apertura: FileNotFoundError, nonostante il controllo")

# TRABOCCHETTO: il controllo non e' sbagliato, e' vecchio. Descrive il disco

# com'era un istante fa; il try descrive il disco nell'istante in cui lo

# usate. Nessun if, per quanto accurato, chiude quell'intervallo.

# Output:

Controllo preventivo: il file c'è

Apertura: FileNotFoundError, nonostante il controllo
```

9.2 Le due difese si sommano: il controllo e il try

```
from pathlib import Path

def leggi_righe(percorso):
    """Restituisce le righe del file, o una lista vuota se non si legge."""
    righe = []

    # 1. Il controllo resta: nel caso piu' frequente da' il messaggio chiaro.
    if not percorso.exists():
        print(f"[!] {percorso.name}: non esiste, controllate il nome")
    else:
        # 2. Il try copre cio' che il controllo non vede. OSError e' la
        # famiglia di 7.3: prende anche la cartella aperta come un file.
        try:
            with open(percorso, encoding="utf-8") as f:
                righe = f.readlines()
        except OSError:
            print(f"[!] {percorso.name}: esiste ma non si legge come file")

    return righe
```

9.2 Tre nomi: il file, il file che manca, la cartella

```
Path("listino_zone.txt").write_text("NORD;8.50\nCENTRO;7.00\n", encoding="utf-8")
```

```
Path("archivio").mkdir()
```

```
for nome in ["listino_zone.txt", "listino_vecchio.txt", "archivio"]:
```

```
    print(f"{nome}: {len(leggi_righe(Path(nome)))} righe")
```

```
# TRABOCCHETTO: con except FileNotFoundError al posto di OSError, la cartella
```

```
#   ferma il programma. E lo ferma con due nomi diversi: IsADirectoryError su
```

```
#   macOS e Linux, PermissionError su Windows. Per questo il messaggio dice
```

```
#   "non si legge come file" e non nomina il tipo dell'eccezione.
```

```
# Output:
```

```
listino_zone.txt: 2 righe
```

```
[!] listino_vecchio.txt: non esiste, controllate il nome
```

```
listino_vecchio.txt: 0 righe
```

```
[!] archivio: esiste ma non si legge come file
```

```
archivio: 0 righe
```

9.3 UnicodeDecodeError: il file in un'altra codifica

```
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe8 in position 44: invalid continuation byte
```

```
position 44    il posto del byte, contato da zero dall'inizio del FILE, non della riga:  
              32 byte della prima riga (a capo compreso), 8 di NS-0602;,  
              4 di Caff, poi la e accentata
```

```
0xe8          in latin-1 la e accentata di Caffè' e' un byte solo
```

```
invalid       in utf-8 0xe8 annuncia altri byte dello stesso carattere,  
continuation  e quello dopo (uno spazio) non lo e'  
byte
```

Il fornitore di NovaStore manda gli ordini di giugno da un gestionale vecchio, salvati in un'altra codifica: la tabella fra caratteri e byte che il Giorno 08 §2.3 vi ha fatto fissare a utf-8. Il rimedio è un secondo tentativo con una codifica di ripiego.

9.3 Il secondo tentativo in latin-1

```
from pathlib import Path

CODIFICA_PRINCIPALE = "utf-8"
CODIFICA_DI_RIPIEGO = "latin-1"

def leggi_testo(percorso):
    """Restituisce il testo del file e la codifica con cui si e' letto."""
    codifica = CODIFICA_PRINCIPALE
    try:
        with open(percorso, encoding=codifica) as f:
            testo = f.read()
    except UnicodeDecodeError:
        # Secondo tentativo. Il primo with ha gia' chiuso il file, anche se
        # si e' interrotto a meta' (lo vedrete al 10.4).
        codifica = CODIFICA_DI_RIPIEGO
        with open(percorso, encoding=codifica) as f:
            testo = f.read()
    return testo, codifica
```

9.3 Due file, due codifiche, lo stesso testo

```
Path("ordini_luglio.txt").write_text("NS-0701;Caffè macinato;6.90\n", encoding="utf-8")
Path("ordini_giugno.txt").write_text(
    "NS-0602;Caffè in capsule;17.50\n", encoding="latin-1"
)
```

```
for nome in ["ordini_luglio.txt", "ordini_giugno.txt"]:
    testo, codifica = leggi_testo(Path(nome))
    print(f"{nome:<19}{codifica:<9}{testo.strip()}")
```

```
# TRABOCCHETTO: l'ordine dei due tentativi non si inverte. latin-1 non
# fallisce MAI, perche' ogni byte e' un suo carattere valido: messa per
# prima leggerebbe anche il file utf-8, e Caffè diventerebbe CaffÃ¨ senza
# un errore. E' la storpiatura silenziosa del Giorno 08 §2.4.
```

```
# Output:
```

```
ordini_luglio.txt  utf-8    NS-0701;Caffè macinato;6.90
ordini_giugno.txt  latin-1  NS-0602;Caffè in capsule;17.50
```

9.3 CHICCA: errors="replace" legge comunque, e lascia un segno

```
from pathlib import Path

Path("ordini_giugno.txt").write_text(
    "NS-0602;Caffè in capsule;17.50\nNS-0603;Tè verde;4.20\n", encoding="latin-1"
)

# Nessuna eccezione: ogni byte incomprensibile diventa U+FFFD.
with open("ordini_giugno.txt", encoding="utf-8", errors="replace") as f:
    testo = f.read()

SOSTITUZIONE = "\ufffd"
print("Caratteri non capiti:", testo.count(SOSTITUZIONE))
for riga in testo.splitlines():
    # Si sostituisce il segno con un ? prima di stampare: il ? passa ovunque.
    print(riga.replace(SOSTITUZIONE, "?"))

# Output:
Caratteri non capiti: 2
NS-0602;Caff? in capsule;17.50
NS-0603;T? verde;4.20
```

9.4 JSONDecodeError: il file malformato

```
import json

from pathlib import Path

# Un file con la virgola mancante a fine riga 2, e un file vuoto.
Path("config_rotto.json").write_text(
    '{\n  "sportelli": 6\n  "attesa_massima": 40\n}\n', encoding="utf-8"
)

Path("config_vuoto.json").write_text("", encoding="utf-8")

for nome in ["config_rotto.json", "config_vuoto.json"]:
    try:
        with open(nome, encoding="utf-8") as f:
            config = json.load(f)

        # Il tipo si scrive con il prefisso del modulo: json.JSONDecodeError.
    except json.JSONDecodeError as e:
        print(f"{nome}: {e}")

# TRABOCCHETTO: scrivete except JSONDecodeError, senza json davanti. Finche'
# i file sono buoni non succede niente, perche' Python valuta il tipo
# dell'except solo quando un'eccezione arriva. Al primo file rotto: un
# NameError dentro la gestione dell'errore, con due traceback in fila.
# Output:
config_rotto.json: Expecting ',' delimiter: line 3 column 3 (char 21)
config_vuoto.json: Expecting value: line 1 column 1 (char 0)
```

9.4 CHICCA: e.lineno, e.colno, e.msg

```
import json

testo = '{\n  "sportelli": 6\n  "attesa_massima": 40\n}\n'

try:
    config = json.loads(testo)
except json.JSONDecodeError as e:
    righe = testo.splitlines()
    print(f"Riga {e.lineno}, colonna {e.colno}: {e.msg}")
    # La riga si prende per indice (Giorno 07 §3.2): lineno parte da 1,
    # la lista da 0.
    print(righe[e.lineno - 1])
    print(" " * (e.colno - 1) + "^")

# Output:
Riga 3, colonna 3: Expecting ',' delimiter
"attesa_massima": 40
^
```

9.5 JSON valido, contenuto sbagliato

```
import json

config = json.loads('{"sportelli": "sei", "sportelli_estivi": true}')

try:
    attesa = config["attesa_massima"]
except KeyError as e:
    print(f"!! manca la chiave {e}: uso 30 minuti")
    attesa = 30

# 2. La chiave c'e', il TIPO no. isinstance(valore, tipo) e' la domanda
#   "questo valore e' di quel tipo?" del Giorno 02 $5, dentro un if.
for chiave in ["sportelli", "sportelli_estivi"]:
    valore = config[chiave]
    if isinstance(valore, int):
        print(f"[OK] {chiave} = {valore}")
    else:
        print(f"!! {chiave} = {valore}: serve un numero intero")

# TRABOCCHETTO: sportelli_estivi vale true, un booleano, e passa. Per Python
#   bool e' un tipo particolare di int (True vale 1): isinstance(True, int)
#   risponde True. Il controllo rigoroso e'
#   isinstance(valore, int) and not isinstance(valore, bool).

print("Attesa:", attesa)
```

9.5 La chiave assente è un'anomalia, non un ripiego

```
[!] manca la chiave 'attesa_massima': uso 30 minuti  
[!] sportelli = sei: serve un numero intero  
[OK] sportelli_estivi = True  
Attesa: 30
```

Gli apici attorno ad `attesa_massima` li mette il `KeyError` stesso (§6.4). La terza riga è il trabocchetto a schermo. E `.get()` qui sarebbe peggio: con la chiave scritta `attesa_masima` nel file, 30 minuti per sempre e nessun avviso.

9.6 La riga che non si converte: il file sporco

```
from pathlib import Path

LISTINO = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}

Path("spedizioni.txt").write_text(
    "LS-2026-MI-0042;NORD;12.5\n"
    "LS-2026-RM-0629;CENTRO;venti\n"
    "LS-2026-NA-0913;SUD;6.7\n"
    "LS-2026-SA-1024;SUD\n"
    "LS-2026-CT-1683;ESTERO;19.7\n"
    "LS-2026-PA-1571;ISOLE;8.5\n"
    "LS-2026-BA-1138;SUD;\n",
    encoding="utf-8",
)

buone = 0

costo_totale = 0.0

scarti = []          # lista di tuple (numero_riga, tipo, riga): Giorno 07 $14.1
scarti_per_tipo = {} # il conta-occorrenze del Giorno 07 $12.2
```

9.6 Saltarla, contarla, segnalarla

```
with open("spedizioni.txt", encoding="utf-8") as f:

    # enumerate(f, 1): il numero di riga come lo conta una persona (Giorno 07 §6.3).

    for numero, riga in enumerate(f, 1):

        riga = riga.strip()

        try:

            # Tre modi di fallire in due righe: campi che mancano (ValueError
            # dall'unpacking), zona fuori listino (KeyError), peso che non e'
            # un numero (ValueError da float).

            codice, zona, peso_testo = riga.split(";")

            costo = LISTINO[zona] * float(peso_testo)

        except (ValueError, KeyError) as e:

            tipo = type(e).__name__

            scarti.append((numero, tipo, riga))

            scarti_per_tipo[tipo] = scarti_per_tipo.get(tipo, 0) + 1

            continue

    buone += 1

    costo_totale += costo
```

9.6 Il riepilogo: le buone e le scartate, riga per riga

```
print(f"Righe buone: {buone}, scartate: {len(scarti)}")

print(f"Costo delle righe buone: {costo_totale:.2f} euro")

for tipo, quanti in scarti_per_tipo.items():
    print(f"  {tipo:<11}{quanti}")

for numero, tipo, riga in scarti:
    print(f"  riga {numero}: {tipo:<11}{riga}")

# TRABOCCHETTO: senza il continue la riga scartata arriva lo stesso a
#   buone += 1, e costo_totale somma il costo della riga PRECEDENTE, ancora
#   in memoria. Risultato: "Righe buone: 7, scartate: 4", cioè undici righe
#   in un file che ne ha sette. Nessun errore: solo conti che non tornano.

# Output:
Righe buone: 3, scartate: 4

Costo delle righe buone: 288.90 euro

ValueError 3

KeyError 1

riga 2: ValueError LS-2026-RM-0629;CENTRO;venti

riga 4: ValueError LS-2026-SA-1024;SUD

riga 5: KeyError LS-2026-CT-1683;ESTERO;19.7

riga 7: ValueError LS-2026-BA-1138;SUD;
```

9.7 csv e le righe corte: il file

```
import csv

from pathlib import Path

Path("ordini.csv").write_text(
    "codice;categoria;importo\n"
    "NS-1001;AUDIO;99.80\n"
    "NS-1002;VIDEO\n"
    "NS-1003;ACCESSORI;\n"
    'NS-1004;"CAVI; ADATTATORI";27.00\n'
    "NS-1005;VIDEO;219.00;omaggio\n",
    encoding="utf-8",
)
```

Il modulo csv del Giorno 08 §9.6 gestisce il campo fra virgolette che contiene il separatore, ma le righe con i campi sbagliati arrivano al vostro unpacking come prima.

9.7 ValueError nell'unpacking e il campo vuoto

```
totale = 0.0

with open("ordini.csv", encoding="utf-8", newline="") as f:

    # L'intestazione si legge a parte con readline(), come nel Giorno 08 $9.5.

    intestazione = f.readline()

    lettore = csv.reader(f, delimiter=";")

    for numero, campi in enumerate(lettore, 2):

        try:

            codice, categoria, importo_testo = campi

            importo = float(importo_testo)

        except ValueError as e:

            # Si stampa il TIPO e il numero di campi, non il messaggio.

            print(f"[!] riga {numero}: {type(e).__name__}, {len(campi)} campi letti")

            continue

        totale += importo

    print(f"    {codice} {categoria:<18}{importo:>8.2f}")

print(f"Totale: {totale:.2f} euro")

# TRABOCCHETTO: il messaggio della riga a quattro campi cambia con la versione
# di Python. 3.12 scrive "too many values to unpack (expected 3)", 3.14
# aggiunge ", got 4". Un output che lo stampa diverge fra due macchine: per
# questo qui si stampa il tipo, e il conteggio lo fa len(campi).
```

9.7 Stesso tipo, tre cause: len(campi) le distingue

```
NS-1001 AUDIO          99.80
[!] riga 3: ValueError, 2 campi letti
[!] riga 4: ValueError, 3 campi letti
    NS-1004 CAVI; ADATTATORI    27.00
[!] riga 6: ValueError, 4 campi letti
Totale: 126.80 euro
```

Campi mancanti, importo vuoto, campo in più: stesso tipo, e len(campi) le distingue. La riga 5 passa: csv l'ha letta in tre campi, dove split(";") ne avrebbe visti quattro.

10. ELSE E FINALLY: IL PERCORSO RIUSCITO E LA PULIZIA

Si pesa il pacco: etichetta se va bene, indietro se no, e il banco si libera sempre

10.1 else dopo except: gira solo se il try è andato bene

```
LISTINO = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}

for zona, peso_testo in [("NORD", "12.5"), ("SUD", "venti"), ("ISOLE", "3")]:
    try:
        # 1. Nel try SOLO la riga che puo' sollevare ValueError.
        peso = float(peso_testo)

    except ValueError:
        # 2. Il pacco torna indietro. peso qui non esiste, e non si usa.
        print(f"zona:<7>{peso_testo:>6} peso non valido, riga scartata")

    else:
        # 3. Il seguito normale: qui peso esiste di sicuro, perche' l'else
        #    gira solo se il try e' arrivato in fondo senza eccezioni.
        costo = peso * LISTINO[zona]
        print(f"zona:<7>{peso_testo:>6} {costo:>7.2f} euro")

# TRABOCCHETTO: l'else si allinea al try e all'except, non all'ultima riga
# rientrata. Scritto quattro spazi piu' a destra diventa un pezzo del
# corpo dell'except, e Python si ferma con SyntaxError prima di partire.

# Output:
NORD      12.5    106.25 euro
SUD      venti  peso non valido, riga scartata
ISOLE       3     42.00 euro
```

10.2 Il calcolo dentro il try: il refuso si nasconde

```
LISTINO_NORD = 8.50

for peso_testo in ["12.5", "venti"]:
    try:
        peso = float(peso_testo)

        costo = peso * LISTINO_NORD

        # Refuso: .2d invece di .2f. E' un difetto del programma, non del dato,
        # e solleva anche lui un ValueError.

        print(f"{peso_testo}: {costo:.2d} euro")
    except ValueError:
        print(f"{peso_testo}: peso non valido, riga scartata")

# TRABOCCHETTO: il peso 12.5 e' perfetto, eppure viene scartato. L'except
# non sa da quale riga arriva il ValueError: per lui "12.5" e "venti" sono
# lo stesso errore, e il refuso resta nascosto per sempre.

# Output:
12.5: peso non valido, riga scartata
venti: peso non valido, riga scartata
```

10.2 Il calcolo nell'else: il try resta stretto

```
LISTINO_NORD = 8.50

for peso_testo in ["12.5", "venti"]:
    try:
        peso = float(peso_testo)
    except ValueError:
        print(f"{peso_testo}: peso non valido, riga scartata")
    else:
        # Il calcolo sta nell'else: fuori dal raggio del try.
        costo = peso * LISTINO_NORD
        print(f"{peso_testo}: {costo:.2d} euro")
```

Output:

Traceback (most recent call last):

File "prova.py", line 11, in <module>

```
print(f"{peso_testo}: {costo:.2d} euro")
```

ValueError: Unknown format code 'd' for object of type 'float'

10.3 finally: il blocco che gira sempre

```
def converti_peso(testo):  
    """Converte un peso; per un testo non numerico usa 0."""  
    try:  
        peso = float(testo)  
    except ValueError:  
        print("  except: peso non valido, uso 0")  
        peso = 0.0  
    finally:  
        # Gira in tutti e tre i casi, anche quando l'eccezione non e' presa.  
        print("  finally: riga chiusa")  
    return peso  
  
for testo in ["12.5", "venti", None]:  
    print(f"Converto {testo}")  
    print("  risultato:", converti_peso(testo))  
  
# TRABOCCHETTO: con None, float() solleva TypeError, e nessun except lo  
# prende. Il finally stampa lo stesso, e SOLO DOPO il programma si ferma.  
# Chi legge l'output vede "riga chiusa" e pensa che sia andato tutto bene:  
# guardate sempre se sotto c'e' un traceback.
```

10.3 Tre percorsi, e finally in tutti e tre

```
Converto 12.5
    finally: riga chiusa
    risultato: 12.5
Converto venti
    except: peso non valido, uso 0
    finally: riga chiusa
    risultato: 0.0
Converto None
    finally: riga chiusa
Traceback (most recent call last):
  File "prova.py", line 16, in <module>
    print(" risultato:", converti_peso(testo))
  File "prova.py", line 4, in converti_peso
    peso = float(testo)
TypeError: float() argument must be a string or a real number, not 'NoneType'
```

Nel terzo percorso la funzione non arriva al return, ma il finally ha fatto il suo lavoro lo stesso.

10.3 CHICCA: finally passa davanti a return

```
def prezzo_zona(zona):  
    listino = {"NORD": 8.50, "SUD": 9.50}  
    try:  
        return listino[zona]  
    finally:  
        print(f" (ricerca di {zona} conclusa)")
```

```
prezzo = prezzo_zona("NORD")  
print("Prezzo ricevuto:", prezzo)
```

Output:

```
(ricerca di NORD conclusa)
```

```
Prezzo ricevuto: 8.5
```

10.4 with: il file si chiude anche quando la lettura si ferma

```
from pathlib import Path

Path("pesi.txt").write_text("12.5\nventi\n6.7\n", encoding="utf-8")

# 1. Con with: l'eccezione esce dal blocco, il file si chiude lo stesso.
try:
    with open("pesi.txt", encoding="utf-8") as f:
        for riga in f:
            print("Peso:", float(riga))
except ValueError as e:
    print("[!] Lettura interrotta:", e)
print("Con with, chiuso?", f.closed)
```

10.4 Senza with: la chiusura la garantisce finally

```
# 2. Senza with: la chiusura la dovete garantire voi, con finally.

g = open("pesi.txt", encoding="utf-8")

try:

    for riga in g:

        print("Peso:", float(riga))

except ValueError:

    print("[!] Lettura interrotta; chiuso?", g.closed)

finally:

    g.close()

print("Dopo il finally, chiuso?", g.closed)


# TRABOCCHETTO: guardate 'venti\n' nel primo messaggio. float() tollera l'a
#   capo attorno a un numero buono (12.5 e' passato), ma quando fallisce lo
#   riporta nel messaggio. E' il \n del Giorno 08 $5.1, reso visibile.


# Output delle due versioni:

Peso: 12.5

[!] Lettura interrotta: could not convert string to float: 'venti\n'

Con with, chiuso? True

Peso: 12.5

[!] Lettura interrotta; chiuso? False

Dopo il finally, chiuso? True
```

10.5 I tre percorsi: li racconta il programma

```
def percorso(testo):  
    """Stampa quali clausole girano per un certo testo in ingresso."""  
    passi = ["try"]  
    try:  
        numero = int(testo)  
    except ValueError:  
        passi.append("except")  
    else:  
        passi.append("else")  
    finally:  
        passi.append("finally")  
    print(f"{str(testo):<7}{' ' -> ' '.join(passi)}")
```

```
percorso("7")
```

```
percorso("sette")
```

```
percorso(None)
```

10.5 Nessun errore, errore catturato, errore non catturato

```
7      try -> else -> finally
sette  try -> except -> finally
None   try -> finally
Traceback (most recent call last):
  File "prova.py", line 17, in <module>
    percorso(None)
  File "prova.py", line 5, in percorso
    numero = int(testo)
TypeError: int() argument must be a string, a bytes-like object or a real number, not 'NoneType'
```

Tre righe, tre percorsi. Nel terzo il finally gira e poi l'eccezione risale: le righe dopo il blocco non vengono eseguite.

10.5 La tabella delle righe eseguite

Percorso	try	except	else	finally	righe dopo il blocco
nessun errore	tutto	no	sì	sì	sì
errore catturato	fino alla riga che solleva	sì	no	sì	sì
errore non catturato	fino alla riga che solleva	no	no	sì	no: l'eccezione risale

È la tabella dei casi limite del Giorno 04 §15.2, compilata dal programma. La colonna finally è l'unica con tre sì.

10.5 Il return nel finally: la forma da non scrivere mai

```
def converti(testo):  
    try:  
        return int(testo)  
    finally:  
        # TRABOCCHETTO: un return nel finally vince su tutto. Sostituisce il  
        # valore del try e INGOLIA l'eccezione: "sette" non da' nessun  
        # errore e restituisce 0, esattamente come "7".  
        return 0
```

```
print(converti("7"), converti("sette"))
```

Output:

0 0

11. RAISE: SOLLEVARE UN'ECCEZIONE

La funzione che non ha una risposta lo dice, invece di inventarla

11.1 La funzione che non sa rispondere: None

```
TARIFFE = [0.00, 12.50, 24.00, 38.00]
```

```
def fascia_isee(isee):  
    """Restituisce la fascia ISEE da 1 a 4."""  
    fascia = None  
    if 0 <= isee <= 8000:  
        fascia = 1  
    elif 8000 < isee <= 16000:  
        fascia = 2  
    elif 16000 < isee <= 28000:  
        fascia = 3  
    elif isee > 28000:  
        fascia = 4  
    return fascia
```

11.1 None arriva al chiamante, e si ferma più in là

```
for isee in [7500, -500]:

    fascia = fascia_isee(isee)

    print(f"ISEE {isee}: fascia {fascia}")

    tariffa = TARIFFE[fascia - 1]

    print(f"    tariffa mensa {tariffa:.2f} euro")


# TRABOCCHETTO: -500 non entra in nessun ramo, e fascia resta None. La
#   f-string lo stampa come la parola None senza protestare, e il programma
#   si ferma solo alla riga dopo, dove None incontra un'operazione vera.


# Output:
ISEE 7500: fascia 1

    tariffa mensa 0.00 euro

ISEE -500: fascia None

Traceback (most recent call last):

  File "prova.py", line 21, in <module>

    tariffa = TARIFFE[fascia - 1]

TypeError: unsupported operand type(s) for -: 'NoneType' and 'int'
```

11.1 La stessa funzione che solleva con raise

```
def fascia_isee(isee):  
    """Restituisce la fascia ISEE da 1 a 4. Solleva ValueError se negativo."""  
    # 1. Il controllo sta in testa: il dato impossibile si ferma qui, prima  
    #    di qualunque calcolo, e con un messaggio che dice perche'.  
    if isee < 0:  
        raise ValueError(f"ISEE negativo ({isee}): un ISEE parte da 0")  
    # 2. Il resto e' la catena crescente con <= del Gruppo 04 §6.5.  
    fascia = 4  
    if isee <= 8000:  
        fascia = 1  
    elif isee <= 16000:  
        fascia = 2  
    elif isee <= 28000:  
        fascia = 3  
    return fascia  
  
print(fascia_isee(16000))  
print(fascia_isee(-500))
```

11.1 Adesso l'ultima riga File è la causa

2

```
Traceback (most recent call last):
```

```
File "prova.py", line 19, in <module>
```

```
    print(fascia_isee(-500))
```

```
File "prova.py", line 6, in fascia_isee
```

```
    raise ValueError(f"ISEE negativo ({isee}): un ISEE parte da 0")
```

```
ValueError: ISEE negativo (-500): un ISEE parte da 0
```

raise ferma la funzione come un return (Giorno 06 §5.3), ma invece di un valore consegna al chiamante un problema. E il traceback punta al punto giusto: la riga 6, dove il dato è stato riconosciuto impossibile.

11.2 Il messaggio: che cosa, dove, che cosa fare

Messaggio	Che cosa dice	Voto
<code>raise ValueError("errore")</code>	niente: c'era già scritto <code>ValueError</code>	da non scrivere
<code>raise ValueError("ISEE non valido")</code>	che cosa	insufficiente
<code>raise ValueError("riga 2: '12.000,50' ha due separatori; togliete il punto delle migliaia")</code>	che cosa, dove, che cosa fare	buono

Il messaggio fra parentesi finirà in fondo a un traceback o sullo schermo di chi usa il programma, e risponde alle tre domande del Giorno 04 §13.8.

11.2 raise ValueError("..."): il messaggio buono al lavoro

```
def converti_isee(testo, numero_riga):  
    """Converte un ISEE scritto all'italiana. Solleva ValueError se non si puo'."""  
  
    # 1. La virgola italiana diventa punto: la pulizia del Giorno 03 $5.4.  
    pulito = testo.strip().replace(",", ".")  
  
    # 2. Due punti vuol dire che c'era il separatore delle migliaia: il caso  
    #    lo conosciamo, e il messaggio lo puo' dire con precisione.  
    if pulito.count(".") > 1:  
        raise ValueError(  
            f"riga {numero_riga}: '{testo}' ha due separatori; "  
            "togliete il punto delle migliaia"  
        )  
  
    # 3. Tutto il resto lo giudica float(), con il suo messaggio.  
    return float(pulito)  
  
for numero, testo in enumerate(["9800,00", "12.000,50", "ventimila"], 1):  
    try:  
        print(f"riga {numero}: {converti_isee(testo, numero):.2f}")  
    except ValueError as e:  
        print(f"[ERRORE] {e}")  
  
# TRABOCCHETTO: il terzo messaggio e' di Python, e non dice ne' la riga ne'  
#    che cosa fare. Non e' sbagliato: e' scritto per un programmatore. Come  
#    tradurlo senza perdere l'originale lo vedete al 11.5.
```

11.2 Un messaggio per l'utente, uno per il programmatore

```
riga 1: 9800.00  
[ERRORE] riga 2: '12.000,50' ha due separatori; togliete il punto delle migliaia  
[ERRORE] could not convert string to float: 'ventimila'
```

Il secondo messaggio l'avete scritto voi e risponde a tutte e tre le domande. Il terzo l'ha scritto Python: dice che cosa, ma non dove né che cosa fare.

11.3 Chi solleva: la funzione valida

```
MESE_MINIMO = 1
```

```
MESE_MASSIMO = 12
```

```
def converti_mese(testo):
```

```
    """Restituisce il mese come intero. Solleva ValueError se non va bene."""
```

```
    # 1. La forma con la guardia del Giorno 04 §13.4: raise non ha bisogno
```

```
    #     di un try, si puo' sollevare anche da un controllo preventivo.
```

```
    if not testo.strip().isdecimal():
```

```
        raise ValueError(f"'{testo}' non è un numero intero")
```

```
    mese = int(testo)
```

```
    # 2. L'intervallo con il confronto concatenato del Giorno 04 §10.3.
```

```
    if not MESE_MINIMO <= mese <= MESE_MASSIMO:
```

```
        raise ValueError(f"{mese} fuori intervallo: da {MESE_MINIMO} a {MESE_MASSIMO}")
```

```
    return mese
```

11.3 Chi cattura: il main() decide

```
def main():  
    # Le risposte che allo sportello arriverebbero dalla tastiera.  
    risposte = ["tre", "0", "4"]  
    mese = 0  
    for tentativo, testo in enumerate(risposte, 1):  
        try:  
            mese = converti_mese(testo)  
        except ValueError as e:  
            # 3. Il main decide: qui si chiede di nuovo.  
            print(f"Tentativo {tentativo}: [!] {e}. Riprovate.")  
            continue  
        print(f"Tentativo {tentativo}: [OK] mese {mese}")  
        break  
    print("Mese registrato:", mese)  
  
if __name__ == "__main__":  
    main()  
  
# TRABOCCHETTO: converti_mese() non stampa niente. Se stampasse lei il  
# messaggio, il file notturno che la usa riempirebbe lo schermo di righe  
# "Riprovate" rivolte a nessuno. Chi valida non parla all'utente.
```

11.3 Tre tentativi, due messaggi della funzione

```
Tentativo 1: [!] 'tre' non è un numero intero. Riprovate.  
Tentativo 2: [!] 0 fuori intervallo: da 1 a 12. Riprovate.  
Tentativo 3: [OK] mese 4  
Mese registrato: 4
```

Il testo dopo [!] l'ha scritto la funzione; il Riprovate l'ha aggiunto il main(). Ognuno dice la parte che sa.

11.4 Il rilancio con raise nudo: annotare e lasciar passare

```
annotazioni = []
```

```
def converti_peso(testo):  
    """Converte un peso. Annota ogni errore e lo lascia proseguire."""  
    try:  
        peso = float(testo)  
    except Exception as e:  
        # 1. Annotare: il tipo e il messaggio, come l'ultima riga del traceback.  
        annotazioni.append(f"{type(e).__name__}: {e}")  
        # 2. Rilanciare: raise nudo rimette in viaggio la STESSA eccezione.  
        raise  
    return peso
```

11.4 Annotato, rilanciato, catturato o no

```
for testo in ["12.5", "venti", None]:

    try:

        print("Peso:", converti_peso(testo))

    except ValueError:

        print(f"[!] {testo}: peso scartato")

    print("  annotazioni finora:", len(annotazioni))

# TRABOCCHETTO: dimenticate il raise. L'errore viene annotato, poi la
# funzione prosegue fino a return peso, e peso non e' mai stato assegnato.
# Il programma si ferma gia' su "venti", con UnboundLocalError: un errore
# nuovo, che nasconde quello vero.

# Output:
Peso: 12.5

    annotazioni finora: 0

[!] venti: peso scartato

    annotazioni finora: 1

Traceback (most recent call last):

  File "prova.py", line 18, in <module>

    print("Peso:", converti_peso(testo))

  File "prova.py", line 7, in converti_peso

    peso = float(testo)

TypeError: float() argument must be a string or a real number, not 'NoneType'
```

11.5 raise dentro except: i due traceback concatenati

```
def converti_isee(testo):  
    """Converte un ISEE scritto all'italiana, con la virgola."""  
    try:  
        isee = float(testo.replace(",", ".", "."))  
    except ValueError:  
        raise ValueError(f"ISEE '{testo}' illeggibile: usate cifre e virgola")  
    return isee  
  
converti_isee("dodicimila")
```

Il §11.2 ha lasciato un problema: il messaggio di float() non dice la riga né che cosa fare. Viene naturale catturarlo e sollevare al suo posto un ValueError con un messaggio migliore. Si può, e Python conserva tutti e due gli errori.

11.5 During handling of the above exception...

```
Traceback (most recent call last):
  File "prova.py", line 4, in converti_isee
    isee = float(testo.replace(",", "."))
ValueError: could not convert string to float: 'dodicimila'
```

During handling of the above exception, another exception occurred:

```
Traceback (most recent call last):
  File "prova.py", line 10, in <module>
    converti_isee("dodicimila")
  File "prova.py", line 6, in converti_isee
    raise ValueError(f"ISEE '{testo}' illeggibile: usate cifre e virgola")
ValueError: ISEE 'dodicimila' illeggibile: usate cifre e virgola
```

La frase in mezzo dice: mentre si gestiva l'eccezione qui sopra, se n'è verificata un'altra. Suona come un incidente, un guasto del gestore. Ma la seconda eccezione è voluta: traduce la prima.

11.5 raise ... from e: la traduzione voluta

```
except ValueError as e:  
    raise ValueError(f"ISEE '{testo}' illeggibile: usate cifre e virgola") from e
```

Il traceback resta identico tranne la frase in mezzo, che diventa:

The above exception was the direct cause of the following exception:

L'eccezione qui sopra è la causa diretta di quella che segue: una traduzione voluta, non un secondo guasto. Si legge dal basso come sempre: prima il messaggio per l'utente, poi, risalendo, il dettaglio tecnico da cui è nato.

11.6 Scegliere il tipo da sollevare

Tipo	Quando	Esempio
ValueError	il tipo è giusto, il valore no	ISEE negativo, mese 13, peso -2
TypeError	il tipo stesso è sbagliato	una zona passata come numero invece che come testo
KeyError	una chiave cercata non esiste	la zona ESTERO nel listino

Il tipo che sollevate è un'informazione per chi cattura: gli dice che genere di problema è, e quindi se è un dato da scartare o un difetto da correggere (§6.8). Tre tipi coprono quasi tutti i casi di oggi.

11.6 TypeError e KeyError, sollevati dalla stessa funzione

```
LISTINO = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}
```

```
def prezzo_zona(zona):  
    """Prezzo al kg della zona. Solleva TypeError o KeyError."""  
  
    # 1. Tipo sbagliato: e' un difetto di chi chiama, non un dato sporco.  
    if not isinstance(zona, str):  
        raise TypeError(f"la zona deve essere un testo, non {type(zona).__name__}")  
  
    # 2. Chiave assente: lo stesso tipo che solleverebbe LISTINO[zona].  
    if zona not in LISTINO:  
        raise KeyError(zona)  
  
    return LISTINO[zona]
```

```
for zona in ["SUD", "ESTERO", 3]:  
    try:  
        print(zona, prezzo_zona(zona))  
    except KeyError as e:  
        print(f"[!] zona {e} fuori listino")  
    except TypeError as e:  
        print(f"[DIFETTO] {e}")
```

```
# TRABOCCHETTO: raise vuole un'eccezione, non una stringa. Scrivete  
# raise "zona sconosciuta" e Python si ferma con un TypeError che parla  
# d'altro: "exceptions must derive from BaseException" (vedi E7).
```

12. LA PROPAGAZIONE: L'ECCEZIONE RISALE LE CHIAMATE

Chi non sa risolvere il problema lo passa più su, finché qualcuno decide

12.1 La risalita: converti() e carica()

```
def converti(campo):  
    print("    converti: inizio")  
    peso = float(campo)  
    print("    converti: fine")  
    return peso
```

```
def carica(righe):  
    print("  carica: inizio")  
    pesi = []  
    for riga in righe:  
        pesi.append(converti(riga))  
    print("  carica: fine")  
    return pesi
```

12.1 ... fino al main(), che cattura

```
def main():  
    print("main: inizio")  
  
    try:  
        pesi = carica(["12.5", "venti"])  
        print("main: pesi caricati", pesi)  
  
    except ValueError as e:  
        print("main: catturato ->", e)  
  
    print("main: fine")  
  
main()  
  
# TRABOCCHETTO: contate le "fine". converti ne stampa una sola su due giri,  
#  carica nessuna, e "pesi caricati" non compare: ogni funzione attraversata  
#  si interrompe nel punto esatto in cui l'eccezione le passa sotto.  
  
# Output:  
main: inizio  
carica: inizio  
converti: inizio  
converti: fine  
converti: inizio  
main: catturato -> could not convert string to float: 'venti'  
main: fine
```

12.2 Dove mettere il try: non dove si decide alla cieca

```
def converti(campo):  
    """Converte un peso. Se il testo non e' un numero... inventa uno zero."""  
    peso = 0.0  
    try:  
        peso = float(campo)  
    except ValueError:  
        print(f"  [!] '{campo}' non valido, uso 0")  
    return peso
```

```
pesi = []  
for riga in ["12.5", "venti", "6.7"]:  
    pesi.append(converti(riga))  
print(f"Colli: {len(pesi)}, peso medio: {sum(pesi) / len(pesi):.2f} kg")  
  
# TRABOCCHETTO: il messaggio c'e', ma il danno e' gia' fatto. Lo zero entra  
#   nella lista come un collo vero, e la media scende da 9.60 a 6.40 kg. E'  
#   il default comodo del Giorno 06 $8.4: un valore che fa girare il  
#   programma e rende falso il risultato.
```

```
# Output:  
[!] 'venti' non valido, uso 0  
Colli: 3, peso medio: 6.40 kg
```

12.2 Il try nel punto che sa che cosa fare

```
def converti(campo):  
    """Converte un peso. Non cattura niente: non sa che cosa farne."""  
    return float(campo)  
  
pesi = []  
for riga in ["12.5", "venti", "6.7"]:  
    try:  
        pesi.append(converti(riga))  
    except ValueError:  
        # Il ciclo SA che cosa fare: la riga si esclude dal conto.  
        print(f"  [!] '{riga}' scartata")  
  
print(f"Colli: {len(pesi)}, peso medio: {sum(pesi) / len(pesi):.2f} kg")  
  
# Output:  
[!] 'venti' scartata  
Colli: 2, peso medio: 9.60 kg
```

12.3 try fuori dal for: il primo errore è un break

```
RIGHE = ["12.5", "3.2", "venti", "6.7", "8.5"]
```

```
def somma_try_fuori(righe):  
    totale = 0.0  
  
    # Il try ABBRACCIA il for: il primo errore esce dal ciclo, come un break.  
    try:  
        for riga in righe:  
            totale += float(riga)  
  
    except ValueError:  
        print(" fuori: file abbandonato alla riga", riga)  
  
    return totale
```

12.3 try dentro il for: l'errore è un continue

```
def somma_try_dentro(righe):  
    totale = 0.0  
  
    # Il try sta DENTRO il for: l'errore salta un giro, come un continue.  
    for riga in righe:  
        try:  
            totale += float(riga)  
        except ValueError:  
            print(" dentro: riga scartata", riga)  
    return totale  
  
print(f"try fuori dal for: {somma_try_fuori(RIGHE):.1f} kg")  
print(f"try dentro il for: {somma_try_dentro(RIGHE):.1f} kg")  
  
# TRABOCCHETTO: la versione "fuori" restituisce comunque un numero, 15.7,  
# che sembra un totale e non lo è: sono solo le righe prima dell'errore.  
# Chi legge il report non ha modo di saperlo, se il messaggio si perde.  
  
# Output:  
fuori: file abbandonato alla riga venti  
try fuori dal for: 15.7 kg  
dentro: riga scartata venti  
try dentro il for: 30.9 kg
```

12.4 Una funzione, due chiamanti: lo sportello

```
PESO_MASSIMO = 30.0
```

```
def converti_peso(testo):  
    """Peso in kg da un testo all'italiana. Solleva ValueError con il motivo."""  
    peso = float(testo.strip().replace(",", "."))  
    if not 0 < peso <= PESO_MASSIMO:  
        raise ValueError(f"{peso} kg fuori dai limiti (0-{PESO_MASSIMO:.0f})")  
    return peso
```

```
# Chiamante 1: lo sportello. Chi sbaglia riprova (le risposte sono simulate).  
for testo in ["venti", "45", "12,5"]:  
    try:  
        peso = converti_peso(testo)  
    except ValueError as e:  
        print(f"Sportello: {e}. Riprovate.")  
    else:  
        print(f"Sportello: accettato {peso} kg")  
        break
```

12.4 Una funzione, due chiamanti: il file notturno

```
# Chiamante 2: il file. La riga sbagliata si conta e si va avanti.

scartate = 0

for testo in ["3.2", "venti", "45", "6,7"]:

    try:

        converti_peso(testo)

    except ValueError:

        scartate += 1

print(f"File: {scartate} righe scartate su 4")


# TRABOCCHETTO: il primo messaggio allo sportello e' in inglese, perche' e'
# quello di float(). Per l'utente va tradotto, con la tecnica del 11.5;
# per il file notturno non serve, perche' il file conta e non parla.


# Output dei due chiamanti:

Sportello: could not convert string to float: 'venti'. Riprovate.

Sportello: 45.0 kg fuori dai limiti (0-30). Riprovate.

Sportello: accettato 12.5 kg

File: 2 righe scartate su 4
```

13. ECCEZIONI CON UN NOME PROPRIO

Due errori da trattare in modo diverso meritano due nomi diversi

13.1 class RigaMalformata(Exception): pass

```
# Due righe che dichiarano un nome nuovo per un genere di errore.

# Exception fra parentesi: la vostra eccezione e' "un tipo di Exception",
# e si comporta in tutto come quelle che conoscete.

class RigaMalformata(Exception):

    pass


errore = RigaMalformata("riga 4: 3 campi attesi, trovati 2")

print(type(errore).__name__)

print(errore)

print(isinstance(errore, Exception))


# TRABOCCHETTO: senza (Exception) la riga si scrive lo stesso, ma quello che
# nasce non e' un'eccezione. Con un messaggio fra parentesi Python si ferma
# gia' alla creazione: "TypeError: RigaMalformata() takes no arguments".
# Senza messaggio, al raise: "exceptions must derive from BaseException".


# Output:

RigaMalformata

riga 4: 3 campi attesi, trovati 2

True
```

13.2 Sollevarla con un messaggio

```
class RigaMalformata(Exception):  
    pass  
  
CAMPI_ATTESI = 3  
  
def leggi_record(numero, riga):  
    """Da riga di testo a (codice, zona, peso). Solleva se la riga non regge."""  
    campi = riga.strip().split(";")  
  
    # 1. Il numero di campi: il controllo del Giorno 08 $9.2, che adesso  
    #     invece di restituire un motivo solleva un errore con un nome.  
    if len(campi) != CAMPI_ATTESI:  
        raise RigaMalformata(  
            f"riga {numero}: {CAMPI_ATTESI} campi attesi, trovati {len(campi)}"  
        )  
  
    # 2. Il peso: qui l'errore lo solleva float(), ed e' un ValueError.  
    return campi[0], campi[1], float(campi[2])
```

13.2 ... e catturarla per nome

```
righe = [  
    "LS-2026-MI-0042;NORD;12.5",  
    "LS-2026-SA-1024;SUD",  
    "LS-2026-NA-0913;SUD;sei",  
]  
  
for numero, riga in enumerate(righe, 1):  
    try:  
        codice, zona, peso = leggi_record(numero, riga)  
        print(f"[OK] {codice} {zona} {peso}")  
    except RigaMalformata as e:  
        print(f"[FORMATO] {e}")  
    except ValueError as e:  
        print(f"[VALORE] riga {numero}: {e}")  
  
# Output:  
[OK] LS-2026-MI-0042 NORD 12.5  
[FORMATO] riga 2: 3 campi attesi, trovati 2  
[VALORE] riga 3: could not convert string to float: 'sei'
```

13.3 Ereditare da ValueError: il codice di prima la cattura

```
class ImportoNonValido(ValueError):  
    pass  
  
def converti_importo(testo):  
    importo = float(testo.replace(",", "."))  
  
    if importo < 0:  
        raise ImportoNonValido(f"importo negativo: {testo}")  
  
    return importo  
  
# Codice scritto PRIMA che la classe esistesse: cattura ValueError e basta.  
for testo in ["19,90", "-5", "dieci"]:  
    try:  
        print(converti_importo(testo))  
    except ValueError as e:  
        print(f"[!] {type(e).__name__}: {e}")  
  
print(issubclass(ImportoNonValido, ValueError))  
  
# TRABOCCHETTO: l'ordine degli except vale anche per le vostre. Un except  
# ValueError scritto PRIMA di un except ImportoNonValido prende tutto, e il  
# secondo non gira mai: e' la regola del 7.2.  
  
# Output:  
  
19.9  
[!] ImportoNonValido: importo negativo: -5  
[!] ValueError: could not convert string to float: 'dieci'  
  
True
```

13.4 Quando non serve: un ValueError con un buon messaggio

```
def converti_importo(testo):  
    """Importo da testo. Solleva ValueError con un messaggio che si capisce."""  
    importo = float(testo.replace(",", "."))  
  
    if importo < 0:  
        raise ValueError(f"importo negativo ({testo}): gli sconti vanno a parte")  
  
    return importo  
  
for testo in ["19,90", "-5"]:  
    try:  
        print(f"{converti_importo(testo):.2f}")  
    except ValueError as e:  
        print(f"[!] {e}")  
  
# TRABOCCHETTO: una classe per ogni errore sembra ordine. Dieci righe di  
# class che nessun except nomina sono dieci nomi da imparare e zero  
# decisioni in piu': il messaggio, qui, fa tutto il lavoro.  
  
# Output:  
19.90  
[!] importo negativo (-5): gli sconti vanno a parte
```

14. ASSERT: IL CONTROLLO CHE IL PROGRAMMATTORE FA SU SE STESSO

Le eccezioni difendono dai dati; assert difende dagli errori vostri

14.1 assert condizione, "messaggio" e AssertionError

```
lette = 7
buone = 4
scartate = 3

# 1. Invariante vera: assert tace, il programma prosegue.
assert lette == buone + scartate, f"lette {lette}, contate {buone + scartate}"
print("Primo controllo superato")

# 2. Simuliamo un difetto nel conteggio: una scartata non e' stata contata.
scartate = 2
assert lette == buone + scartate, f"lette {lette}, contate {buone + scartate}"
print("Secondo controllo superato")

# Output:
Primo controllo superato
Traceback (most recent call last):
  File "prova.py", line 11, in <module>
    assert lette == buone + scartate, f"lette {lette}, contate {buone + scartate}"
AssertionError: lette 7, contate 6
```

14.2 assert non valida l'input: si può spegnere

```
eta = -3
```

```
# TRABOCCHETTO: con le parentesi, assert riceve UNA cosa sola, la coppia
```

```
# (condizione, messaggio). Una coppia non vuota vale sempre True (la
```

```
# chicca del Giorno 07 §9.3), quindi questa assert non scatta MAI.
```

```
assert (eta >= 0, "eta' negativa")
```

```
print("L'assert fra parentesi non ha fermato niente: eta =", eta)
```

```
# La validazione vera, che nessuna opzione di avvio puo' spegnere.
```

```
if eta < 0:
```

```
    print("[ERRORE] Età: serve un numero da 0 in su")
```

```
# Output:
```

```
L'assert fra parentesi non ha fermato niente: eta = -3
```

```
[ERRORE] Età: serve un numero da 0 in su
```

14.3 Tre assert sotto una funzione: il collaudo a mano

```
def fascia_isee(isee):  
    """Restituisce la fascia ISEE da 1 a 4 (soglie del Giorno 04 §6.5)."""  
  
    fascia = 4  
  
    if isee <= 8000:  
        fascia = 1  
  
    elif isee <= 16000:  
        fascia = 2  
  
    elif isee <= 28000:  
        fascia = 3  
  
    return fascia
```

```
# Collaudo sui bordi: il valore esatto di ogni soglia e il centesimo dopo.  
assert fascia_isee(8000) == 1, "8000 esatti: fascia 1"  
  
assert fascia_isee(8000.01) == 2, "8000,01: fascia 2"  
  
assert fascia_isee(16000) == 2, "16000 esatti: fascia 2"  
  
assert fascia_isee(28000) == 3, "28000 esatti: fascia 3"  
  
assert fascia_isee(28000.01) == 4, "28000,01: fascia 4"  
  
print("Collaudo dei bordi: 5 casi su 5")
```

```
# TRABOCCHETTO: cambiate il primo <= in <. Il programma non da' nessun  
# errore di Python, ma la prima assert si ferma con  
# "AssertionError: 8000 esatti: fascia 1": e' il bordo del Giorno 04 §6.3,  
# trovato dal programma invece che dall'ufficio.
```

14.4 L'invariante: righe lette = buone + scartate

```
from pathlib import Path

Path("pesi.txt").write_text("12.5\nventi\n\n6.7\n", encoding="utf-8")

lette = buone = scartate = 0

with open("pesi.txt", encoding="utf-8") as f:
    for riga in f:
        lette += 1

        # La riga vuota si salta, come nel Giorno 08 $5.3...

        if riga.strip() == "":
            continue

        try:
            float(riga)
        except ValueError:
            scartate += 1
        else:
            buone += 1

print(f"Lette {lette}, buone {buone}, scartate {scartate}")

# ... ma non finisce in nessuna colonna: l'invariante se ne accorge.

assert lette == buone + scartate, f"lette {lette}, contate {buone + scartate}"
```

14.4 L'assert scatta: il difetto è nel programma

```
Lette 4, buone 2, scartate 1
Traceback (most recent call last):
  File "prova.py", line 21, in <module>
    assert lette == buone + scartate, f"lette {lette}, contate {buone + scartate}"
AssertionError: lette 4, contate 3
```

15. IL REGISTRO DEGLI ERRORI SU FILE

Il programma notturno scarta sei righe: chi le legge, se lo schermo è spento?

15.1 Scrivere l'errore invece di stamparlo: la modalità a

```
from pathlib import Path

PERCORSO_REGISTRO = Path("registro_errori.txt")

righe = ["12.5", "venti", "6.7", ""]
buone = 0

# 1. Ripartenza pulita: il registro si svuota all'avvio (lo spiega il 15.3).
PERCORSO_REGISTRO.write_text("", encoding="utf-8")

# 2. Modalita' "a": ogni riga si aggiunge in fondo, niente si cancella.
with open(PERCORSO_REGISTRO, "a", encoding="utf-8") as registro:

    for numero, riga in enumerate(righe, 1):

        try:

            float(riga)

        except ValueError as e:

            # 3. Il dettaglio va nel registro, non sullo schermo.

            print(f"riga {numero}: {e}", file=registro)

        else:

            buone += 1
```

15.1 A schermo il riepilogo, nel registro il dettaglio

```
# 4. A schermo, per chi usa il programma: il riepilogo e dove guardare.  
print(f"Righe buone: {buone} su {len(righe)}. Dettagli in {PERCORSO_REGISTRO.name}")  
print("--- contenuto del registro ---")  
print(PERCORSO_REGISTRO.read_text(encoding="utf-8"), end="")
```

```
# TRABOCCHETTO: dimenticate file=registro e la print va sullo schermo, come  
# sempre. Nessun errore: il registro resta vuoto, e ve ne accorgete la  
# mattina in cui vi serve.
```

```
# Output:  
Righe buone: 2 su 4. Dettagli in registro_errori.txt  
--- contenuto del registro ---  
riga 2: could not convert string to float: 'venti'  
riga 4: could not convert string to float: ''
```

15.2 Una riga per errore: numero, tipo, messaggio, dato

```
from pathlib import Path

PERCORSO_REGISTRO = Path("registro_spedizioni.txt")

LISTINO = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}

RIGHE = [

    "LS-0042;NORD;12.5",

    "LS-1024;SUD",

    "LS-1683;ESTERO;19.7",

    "LS-0629;CENTRO;venti",

]

with open(PERCORSO_REGISTRO, "w", encoding="utf-8") as registro:

    for numero, riga in enumerate(RIGHE, 1):

        try:

            codice, zona, peso = riga.split(";")

            costo = LISTINO[zona] * float(peso)

        except (ValueError, KeyError) as e:

            # Quattro pezzi uniti da ;, con il dato originale PER ULTIMO:

            # e' l'unico che puo' contenere a sua volta dei punto e virgola.

            print(numero, type(e).__name__, e, riga, sep=";", file=registro)

print(PERCORSO_REGISTRO.read_text(encoding="utf-8"), end="")
```

15.2 Rileggere il registro: split(";", 3)

```
# Rilettura con il lettore di file delimitati del Giorno 08 §9.1.

with open(PERCORSO_REGISTRO, encoding="utf-8") as f:

    for riga in f:

        # split(";", 3): al massimo tre tagli, il quarto pezzo resta intero.

        numero, tipo, messaggio, dato = riga.rstrip("\n").split(";", 3)

        print(f"riga {numero:>2}  {tipo:<11}{dato}")


# TRABOCCHETTO: con split(";") senza il 3, la prima riga del registro si
# spezza in cinque pezzi (il dato LS-1024;SUD ne vale due) e l'unpacking a
# quattro nomi si ferma con ValueError. Il registro degli errori che
# solleva un errore: il limite del dato in coda vale solo con il 3.


# Output delle due slide:

2;ValueError;not enough values to unpack (expected 3, got 2);LS-1024;SUD

3;KeyError;'ESTERO';LS-1683;ESTERO;19.7

4;ValueError;could not convert string to float: 'venti';LS-0629;CENTRO;venti

riga  2  ValueError LS-1024;SUD

riga  3  KeyError   LS-1683;ESTERO;19.7

riga  4  ValueError LS-0629;CENTRO;venti
```

15.3 Il registro che riparte pulito: w all'avvio, a durante

```
from pathlib import Path

PERCORSO_REGISTRO = Path("registro.txt")

def registra(messaggio):
    """Aggiunge una riga al registro: modalita' a, una riga per evento."""
    with open(PERCORSO_REGISTRO, "a", encoding="utf-8") as registro:
        print(messaggio, file=registro)

def esegui(righe, riparti_pulito):
    """Una esecuzione del programma: registra le righe non numeriche."""
    if riparti_pulito:
        # write_text apre in "w": il file si svuota prima di cominciare.
        PERCORSO_REGISTRO.write_text("", encoding="utf-8")
    for riga in righe:
        try:
            float(riga)
        except ValueError:
            registra(f"scartata: {riga}")
```

15.3 Due esecuzioni di fila, con e senza azzeramento

```
def righe_nel_registro():  
    with open(PERCORSO_REGISTRO, encoding="utf-8") as f:  
        return len(f.readlines())  
  
for giro in [1, 2]:  
    esegui(["12.5", "venti", "sei"], riparti_pulito=False)  
    print(f"solo 'a',      esecuzione {giro}: {righe_nel_registro()} righe")  
for giro in [1, 2]:  
    esegui(["12.5", "venti", "sei"], riparti_pulito=True)  
    print(f"'w' poi 'a',  esecuzione {giro}: {righe_nel_registro()} righe")  
  
# TRABOCCHETTO: il difetto del registro senza azzeramento non si vede alla  
# prima esecuzione, che e' quella che collaudate. Si vede alla seconda,  
# quando gli errori raddoppiano senza che i dati siano cambiati.  
  
# Output:  
solo 'a',      esecuzione 1: 2 righe  
solo 'a',      esecuzione 2: 4 righe  
'w' poi 'a',  esecuzione 1: 2 righe  
'w' poi 'a',  esecuzione 2: 2 righe
```

15.4 Il registro di una notte, nel formato del 15.2

```
from pathlib import Path

PERCORSO_REGISTRO = Path("registro_spedizioni.txt")

# Il registro di una notte, nel formato del 15.2.

PERCORSO_REGISTRO.write_text(
    "2;ValueError;not enough values to unpack (expected 3, got 2);LS-1024;SUD\n"
    "3;KeyError;'ESTERO';LS-1683;ESTERO;19.7\n"
    "4;ValueError;could not convert string to float: 'venti';LS-0629;CENTRO;venti\n"
    "7;ValueError;could not convert string to float: '';LS-1138;SUD;\n",
    encoding="utf-8",
)
```

Il registro è un file di dati come gli altri, e i programmi del Giorno 07 ci lavorano sopra senza cambiare una riga. Il responsabile chiede: quanti errori di ciascun tipo, stanotte?

15.4 Il conta-occorrenze sul registro

```
conteggi = {}

with open(PERCORSO_REGISTRO, encoding="utf-8") as f:

    for riga in f:

        numero, tipo, messaggio, dato = riga.rstrip("\n").split(";", 3)

        conteggi[tipo] = conteggi.get(tipo, 0) + 1

# Il denominatore si calcola una volta, fuori dal ciclo (Giorno 07 $12.3).
totale = sum(conteggi.values())

for tipo, quanti in conteggi.items():

    print(f" {tipo:<11}{quanti:>2} {quanti / totale:>5.1%} {'#' * quanti}")

# TRABOCCHETTO: len(conteggi) vale 2 (i TIPI), sum(conteggi.values()) vale
# 4 (gli ERRORI). Con il primo come denominatore le quote sommano a 200%:
# e' lo stesso errore del Giorno 07, su un file diverso.

# Output:
ValueError 3 75.0% ###
KeyError 1 25.0% #
```

16. UN PROGRAMMA ROBUSTO, DALL'INIZIO ALLA FINE

Arriva in fondo, e dice che cosa ha scartato e perché

16.1 Lo schema in quattro fasi: carica, valida, elabora, riporta

Fase	Che cosa fa	Strumento per gli errori	Dove l'avete visto
carica	dal disco alla memoria	try sui file, except OSError / ValueError	cap. 9
valida	da testo a dato	raise con messaggio, nome proprio se serve	cap. 11, 13
elabora	conti su dati già puliti	nessun try: qui un errore è un difetto	cap. 16.3
riporta	schermo, file, registro	registro in a, assert sull'invariante	cap. 14, 15

Il ciclo di vita di un dato del Giorno 03 §12.1 e lo schema del Giorno 08 §14.1 hanno le stesse quattro fasi. Oggi ogni fase ha anche il suo strumento per gli errori. Il programma del Giorno 08 si fermava al primo imprevisto; questo arriva in fondo.

16.1 Lo scheletro: carica e valida

```
def carica():  
    """FASE 1: dal disco alla memoria. Qui vivono i try sui file (cap. 9)."""  
    return ["12.5", "venti", "6.7"]  
  
def valida(testo):  
    """FASE 2: da testo a dato. Qui vive il raise (cap. 11)."""  
    peso = float(testo)  
    if peso <= 0:  
        raise ValueError(f"peso {peso} non positivo")  
    return peso
```

Lo scheletro, prima del programma vero. Ogni funzione porta nella docstring la sua fase e il capitolo che le dà lo strumento.

16.1 Lo scheletro: il main() fa il regista

```
def main():  
    righe = carica()  
  
    pesi = []  
  
    scartate = 0  
  
    for testo in righe:  
        # Il try sta nel ciclo, che sa che cosa fare di una riga cattiva (12.2).  
  
        try:  
            pesi.append(valida(testo))  
  
        except ValueError:  
            scartate += 1  
  
    # FASE 3 - ELABORA: niente try, solo conti su dati gia' puliti.  
    totale = sum(pesi)  
  
    # FASE 4 - RIPORTA, con l'invariante in coda (cap. 14).  
    assert len(pesi) + scartate == len(righe), "conteggio incoerente"  
    print(f"Colli {len(pesi)}, kg {totale:.1f}, scartate {scartate}")  
  
if __name__ == "__main__":  
    main()  
  
# Output:  
Colli 2, kg 19.2, scartate 1
```

16.2 Il programma completo: costanti e file di prova

```
import json

from pathlib import Path

PERCORSO_LISTINO = Path("listino.json")
PERCORSO_SPEDIZIONI = Path("spedizioni_lotto.txt")
PERCORSO_REGISTRO = Path("registro_lotto.txt")
CAMPI_ATTESI = 3


class RigaMalformata(Exception):
    pass


def prepara_file_di_prova():
    """Crea i file di partenza: nel programma vero li manda LogiSud."""
    PERCORSO_LISTINO.write_text(
        '{"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}\n',
        encoding="utf-8",
    )
```

16.2 Il lotto sporco, e il caricamento del listino

```
PERCORSO_SPEDIZIONI.write_text(
    "LS-2026-VR-0301;NORD;10.0\n"
    "LS-2026-BG-0302;nord ;4,5\n"
    "LS-2026-PG-0303;CENTRO;7.5\n"
    "LS-2026-AN-0304;CENTRO;sette\n"
    "\n"
    "LS-2026-BR-0305;SUD;12.0\n"
    "LS-2026-TA-0306;SUD\n"
    "LS-2026-ME-0307;ISOLE;3.0\n"
    "LS-2026-ZH-0308;ESTERO;5.0\n"
    "LS-2026-OT-0309;ISOLE;-2.0\n",
    encoding="utf-8",
)
```

```
def carica_listino(percorso):
    """FASE 1. Restituisce il listino; gli errori del file risalgono."""
    with open(percorso, encoding="utf-8") as f:
        return json.load(f)
```

16.2 Valida: leggi_record() solleva, registra() annota

```
def leggi_record(riga, listino):  
    """FASE 2. Da riga a (zona, peso). Solleva se la riga non regge."""  
  
    campi = riga.split(";")  
  
    if len(campi) != CAMPI_ATTESI:  
        raise RigaMalformata(f"{len(campi)} campi invece di {CAMPI_ATTESI}")  
  
    codice, zona, peso_testo = campi  
  
    # La prevenzione del Giorno 03 viene PRIMA: spazi, minuscole, virgola.  
  
    zona = zona.strip().upper()  
  
    if zona not in listino:  
        raise KeyError(zona)  
  
    peso = float(peso_testo.strip().replace(",", "."))  
  
    if peso <= 0:  
        raise ValueError(f"peso {peso} non positivo")  
  
    return zona, peso  
  
  
def registra(numero, errore, riga):  
    """Una riga di registro nel formato del 15.2."""  
  
    with open(PERCORSO_REGISTRO, "a", encoding="utf-8") as registro:  
        print(numero, type(errore).__name__, errore, riga, sep=";", file=registro)
```

16.2 Carica: senza listino si esce

```
def main():  
    # FASE 1 - CARICA. Senza listino non c'e' niente da calcolare: si esce.  
  
    try:  
        listino = carica_listino(PERCORSO_LISTINO)  
    except (OSError, ValueError) as e:  
        print(f"[ERRORE] listino non utilizzabile ({type(e).__name__})")  
        return  
  
    PERCORSO_REGISTRO.write_text("", encoding="utf-8")  
  
    # FASI 2 e 3 - VALIDA ed ELABORA, riga per riga: il try sta nel ciclo.  
  
    lette = buone = scartate = vuote = 0  
  
    colli = {}  
  
    kg = {}
```

16.2 Valida ed elabora: il try dentro il ciclo

```
with open(PERCORSO_SPEDIZIONI, encoding="utf-8") as f:
    for numero, riga in enumerate(f, 1):
        lette += 1
        riga = riga.strip()
        if riga == "":
            vuote += 1
            continue
        try:
            zona, peso = leggi_record(riga, listino)
        except (RigaMalformata, KeyError, ValueError) as e:
            scartate += 1
            registra(numero, e, riga)
            continue
        buone += 1
        colli[zona] = colli.get(zona, 0) + 1
        kg[zona] = kg.get(zona, 0) + peso
```

16.2 Riporta: la tabella, l'invariante, il registro

```
# FASE 4 - RIPORTA, con la tabella a larghezza fissa del Giorno 03 $9.

print(f"{'ZONA':<8}{'COLLI':>6}{'KG':>8}{'COSTO':>10}")

totale = 0.0

for zona in colli:

    costo = kg[zona] * listino[zona]

    totale += costo

    print(f"{zona:<8}{colli[zona]:>6}{kg[zona]:>8.1f}{costo:>10.2f}")

print(f"{'TOTALE':<8}{buone:>6}{sum(kg.values()):>8.1f}{totale:>10.2f}")

print(f"Righe: {lette} lette, {buone} buone, {scartate} scartate, {vuote} vuote")

# L'invariante del 14.4, questa volta a tre termini.

assert lette == buone + scartate + vuote, "conteggio delle righe incoerente"

print(f"Scarti registrati in {PERCORSO_REGISTRO.name}:")

print(PERCORSO_REGISTRO.read_text(encoding="utf-8"), end="")
```

```
if __name__ == "__main__":

    prepara_file_di_prova()

    main()
```

16.2 Il report esce, e dice che cosa ha scartato

ZONA	COLLI	KG	COSTO
NORD	2	14.5	123.25
CENTRO	1	7.5	52.50
SUD	1	12.0	114.00
ISOLE	1	3.0	42.00
TOTALE	5	37.0	331.75

Righe: 10 lette, 5 buone, 4 scartate, 1 vuote

Scarti registrati in registro_lotto.txt:

```
4;ValueError;could not convert string to float: 'sette';LS-2026-AN-0304;CENTRO;sette
7;RigaMalformata;2 campi invece di 3;LS-2026-TA-0306;SUD
9;KeyError;'ESTERO';LS-2026-ZH-0308;ESTERO;5.0
10;ValueError;peso -2.0 non positivo;LS-2026-OT-0309;ISOLE;-2.0
```

Leggetelo per fasi, non per righe. NORD ha due colli: la riga 2, con nord minuscolo e la virgola italiana, passa, perché la prevenzione del Giorno 03 lavora prima del try. Quattro scarti, tre tipi, e ognuno con il suo dato originale in coda.

16.3 Che cosa il try non deve fare: nascondere un errore logico

```
def media_pesi(pesi):  
    """Peso medio. Non si ferma mai, e per questo mente."""  
    try:  
        media = sum(pesi) / len(pesi)  
    except Exception:  
        media = 0.0  
    return media  
  
# Un peso rimasto testo: e' un difetto a monte, nel caricamento.  
pesi_nord = [12.5, 3.2, "6.7"]  
print(f"Peso medio NORD: {media_pesi(pesi_nord):.2f} kg")  
print(f"Peso medio SUD: {media_pesi([]):.2f} kg")  
  
# TRABOCCHETTO: due problemi opposti, la stessa risposta. NORD ha un difetto  
# nel programma (un TypeError nascosto), SUD non ha dati (una divisione per  
# zero nascosta). Il report dice 0.00 per entrambi, e 0.00 sembra un numero.  
  
# Output:  
Peso medio NORD: 0.00 kg  
Peso medio SUD: 0.00 kg
```

16.3 La versione onesta: il difetto si mostra

```
def media_pesi(pesi):  
    """Peso medio. Solleva ValueError se non ci sono pesi."""  
    if len(pesi) == 0:  
        raise ValueError("nessun peso: la media non esiste")  
    return sum(pesi) / len(pesi)
```

```
pesi_nord = [12.5, 3.2, "6.7"]  
print(f"Peso medio NORD: {media_pesi(pesi_nord):.2f} kg")
```

Output:

Traceback (most recent call last):

```
File "prova.py", line 9, in <module>  
    print(f"Peso medio NORD: {media_pesi(pesi_nord):.2f} kg")  
File "prova.py", line 5, in media_pesi  
    return sum(pesi) / len(pesi)
```

TypeError: unsupported operand type(s) for +: 'float' and 'str'

17. ERRORI TIPICI DELLA GIORNATA

Quasi tutti non danno errore: il try fa sparire il traceback che doveva restare

E1 — except: nudo che ingoia un refuso

```
totale = 0

for testo in ["12", "8", "venti"]:
    try:
        totale = totlae + int(testo)
    except:
        print(f"[!] '{testo}' non è un numero")

print("Totale:", totale)

# TRABOCCHETTO: totlae e' un refuso, e solleva NameError a ogni giro.
# L'except senza tipo lo prende e lo chiama "non e' un numero": anche 12
# e 8, che sono numeri perfetti. Con except ValueError il refuso avrebbe
# fermato il programma al primo giro, con il nome sbagliato in chiaro.

# Output:
[!] '12' non è un numero
[!] '8' non è un numero
[!] 'venti' non è un numero
Totale: 0
```

E2 — Il try da trenta righe: cattura l'eccezione sbagliata

```
CATEGORIE = ["AUDIO", "VIDEO", "ACCESSORI"]

for quantita_testo, categoria in [("2", "AUDIO"), ("3", "CAVI")]:
    try:
        quantita = int(quantita_testo)

        posizione = CATEGORIE.index(categoria)

        print(f"{categoria}: {quantita} pezzi, colonna {posizione}")
    except ValueError:
        print(f"[!] quantità '{quantita_testo}' non valida")

# TRABOCCHETTO: anche .index() su un elemento assente solleva ValueError
# (Giorno 07 §5.2). Il messaggio accusa la quantità 3, che è giusta, e
# tace sulla categoria CAVI, che è il problema vero.

# Output:
AUDIO: 2 pezzi, colonna 0
[!] quantità '3' non valida
```

E3 — except Exception prima di except ValueError

```
for testo in ["12", "venti"]:
    try:
        print(int(testo) * 2)
    except Exception:
        print("[ERRORE] qualcosa non va")
    except ValueError:
        print(f"[!] '{testo}' non è un intero: riscrivetelo in cifre")

# TRABOCCHETTO: Python prova gli except dall'alto e si ferma al primo che
#   combacia. Exception combacia con tutto, e il messaggio utile, scritto
#   con cura due righe sotto, non si stampera' mai. Nessun avviso.

# Output:
24
[ERRORE] qualcosa non va
```

E4 — La variabile del try usata dopo l'except

```
def leggi_eta(testo):  
    try:  
        eta = int(testo)  
    except ValueError:  
        print(f"[!] '{testo}' non è un'età")  
    return eta  
  
print(leggi_eta("41"))  
print(leggi_eta("quaranta"))  
  
# TRABOCCHETTO: sul ramo dell'errore eta non e' mai stata assegnata. Dentro  
# una funzione il messaggio e' UnboundLocalError; fuori da ogni funzione  
# sarebbe NameError. Stesso difetto, due nomi (Giorno 06 $11.4).  
  
# Output:  
41  
[!] 'quaranta' non è un'età  
Traceback (most recent call last):  
  File "prova.py", line 10, in <module>  
    print(leggi_eta("quaranta"))  
  File "prova.py", line 6, in leggi_eta  
    return eta  
UnboundLocalError: cannot access local variable 'eta' where it is not associated with a value
```

E5 — except ...: pass, l'errore sparito senza traccia

```
pesi = []

for testo in ["12.5", "12,5", "6.7"]:
    try:
        pesi.append(float(testo))
    except ValueError:
        pass

print(f"Colli: {len(pesi)}, totale {sum(pesi):.1f} kg")

# TRABOCCHETTO: tre colli in ingresso, due nel totale, zero messaggi. Il
# collo perso ha la virgola italiana: la prevenzione del Giorno 03 lo
# avrebbe salvato, il pass lo ha fatto sparire.

# Output:
Colli: 2, totale 19.2 kg
```

E6 — e usato dopo l'except: il nome che Python cancella

```
try:
    peso = float("venti")
except ValueError as e:
    print("[!] peso non valido")

print("Dettaglio:", e)

# TRABOCCHETTO: il nome dopo "as" vive solo dentro il suo except. All'uscita
# Python lo cancella, anche se in apparenza e' una variabile come le altre.

# Output:
[!] peso non valido

Traceback (most recent call last):
  File "prova.py", line 6, in <module>
    print("Dettaglio:", e)
NameError: name 'e' is not defined
```

E7 — raise "testo": exceptions must derive from BaseException

```
peso = -2.0

if peso <= 0:
    raise "peso non positivo"

# TRABOCCHETTO: raise vuole un'eccezione, non una stringa. Il TypeError che
#   ne esce parla di "BaseException" e non del peso: chi legge cerca il
#   problema nel dato, e il problema e' la riga del raise.

# Output:
Traceback (most recent call last):
  File "prova.py", line 3, in <module>
    raise "peso non positivo"
TypeError: exceptions must derive from BaseException
```

E8 — assert sull'input dell'utente

```
def registra_eta(testo):  
    """Registra un'eta'. La validazione e' un if con raise, non un assert."""  
    eta = int(testo)  
    if eta < 0:  
        raise ValueError(f"età {eta}: serve un numero da 0 in su")  
    return eta  
  
for testo in ["41", "-3"]:  
    try:  
        print("Registrata:", registra_eta(testo))  
    except ValueError as e:  
        print("[ERRORE]", e)  
  
# TRABOCCHETTO: con assert eta >= 0 al posto dell'if, chi avvia il programma  
#   con python -O vede registrata un'eta' di -3. E con le parentesi, assert  
#   (eta >= 0, "...") non scatta nemmeno senza -O ($14.2).  
  
# Output:  
Registrata: 41  
[ERRORE] età -3: serve un numero da 0 in su
```

E9 — Il percorso assoluto nel messaggio

```
from pathlib import Path

percorso = Path("ordini_ottobre.txt").resolve()

print("Percorso assoluto?", percorso.is_absolute())

if not percorso.exists():

    print(f"[ERRORE] manca {percorso.name}: chiedetelo al fornitore")

# TRABOCCHETTO: con f"{percorso}" al posto di percorso.name il messaggio
#   conterrebbe /Users/... su un Mac e C:\Users\... su Windows: diverso su
#   ogni macchina, illeggibile per l'utente, impossibile da collaudare.

# Output:

Percorso assoluto? True

[ERRORE] manca ordini_ottobre.txt: chiedetelo al fornitore
```

RIEPILOGO DEL GIORNO

Sedici righe da riguardare stasera

Riepilogo del giorno (1/2)

Eccezione: un evento che interrompe il flusso. Un oggetto con un tipo e un messaggio.

La sintassi sbagliata non è un'eccezione in corsa: il file non parte nemmeno.

Traceback: dal basso. Tipo, messaggio, poi le righe File dalla più interna alla più esterna.

Il dato fra apici è quello esatto. La riga indicata è dove il sintomo si vede.

try / except: il salto è solo in avanti. Nel try una riga, non trenta.

Il tipo dopo except dichiara che cosa vi aspettate. Vince il primo che combacia.

as e per il messaggio, `type(e).__name__` per il tipo. Mai l'except nudo.

Catalogo: quelle da dati si catturano, quelle da difetto si correggono.

Gerarchia: `LookupError`, `ArithmeticError`, `OSError`, `ValueError` con codifica e JSON.

Except dal particolare al generale. Ctrl+C sta fuori da Exception.

Prevenire con l'if se il caso è prevedibile; gestire con il try se decide un parser

o il mondo esterno; sul dato umano tutti e due: try per la forma, if per l'intervallo.

File: `FileNotFoundError`, `UnicodeDecodeError`, `JSONDecodeError` con riga e colonna.

Il file che sparisce fra `exists()` e `open()` lo copre solo un try.

Riepilogo del giorno (2/2)

else per il percorso riuscito, fuori dal try stretto; finally gira comunque.

with è un finally già scritto.

raise: la funzione che non sa rispondere solleva con un messaggio che dice

che cosa, dove e che cosa fare. Chi solleva non stampa: decide chi chiama.

Propagazione: il try va dove si sa che cosa fare. Dentro il for, una riga persa;

fuori, tutto il file perso.

class NomeErrore(Exception): pass dà un nome, e serve solo se un except lo distingue.

assert controlla il programma, non l'input: si può spegnere con python -O.

Registro: w all'avvio, a per ogni errore. Numero, tipo, messaggio, dato originale.

Programma robusto: carica, valida, elabora, riporta. Il try non nasconde errori logici.

Errori tipici: except nudo, try largo, generale per primo, variabile del try usata

dopo, except: pass, raise "testo", assert sull'input, percorso assoluto nel messaggio.

Grazie.

shaping the skills of tomorrow

